CoolProp CAPE-OPEN Property Package: Technical Reference

Release 0.4

Anders Andreasen

Oct 27, 2026

Part I: Foundations

I Foundations 2

1 Introduction 3
- 1.1 Purpose and scope . . . 3
- 1.2 What made this possible: the upstream mixture flash . . . 3
- 1.3 What this reference covers . . . 4
- 1.4 Audience . . . 4

2 Architecture 5
- 2.1 The adapter pattern . . . 5
- 2.2 The AbstractState handle model . . . 6
- 2.3 Pure-fluid routing . . . 7
- 2.4 Single-phase property fallbacks . . . 7
- 2.5 COM surface and lifetime . . . 7
- 2.6 Configuration GUI . . . 8
- 2.7 Registered as a selectable property package . . . 8
- 2.8 Persistence . . . 8

II Implementation and Design Decisions 12

3 Design Decisions 13
- 3.1 Zero-fraction components . . . 13
- 3.2 Single-component (pure) compositions . . . 14
- 3.3 Compound-order alignment to the Material Object . . . 14
- 3.4 Equation-of-state / backend selection . . . 15
- 3.5 PH/PS flash strategy . . . 16

4 Flash and Property Routing 18
- 4.1 Flash routing overview . . . 18
- 4.2 TP on the saturation line . . . 19
- 4.3 The C# PH/PS flash . . . 19
- 4.4 The native PH/PS flash (opt-in) . . . 21
- 4.5 Micro-perturbation convergence retry . . . 22

5 Transport-Property Fallbacks 23
- 5.1 Gas Viscosity . . . 24
- 5.2 Liquid Viscosity . . . 24
- 5.3 Thermal Conductivity (Gharagheizi 2013) . . . 25
- 5.4 Surface Tension . . . 26

5.5 Component normal boiling point . . . 26

6 The Incompressible Companion Package **27**
6.1 Scope . . . 27
6.2 Why mass basis only . . . 27
6.3 Low-level construction . . . 28
6.4 Flashes supported . . . 28
6.5 Freezing point . . . 28
6.6 Configuration GUI . . . 28

III Validation **30**

7 Validation **31**
7.1 CHP: pure water, single-component routing . . . 31
7.2 TEALARC: dual mixed refrigerant, the C2/C3 case study . . . 33
7.3 CO2-from-NG: extractive distillation, a hard mixture and a cold-start caveat . . . 34
7.4 Snøhvit: single mixed refrigerant, multi-stage compression . . . 37
7.5 Air separation unit (ASU): cryogenic N2/O2/Ar distillation . . . 38
7.6 CO_2 compression, dehydration and liquefaction (CCS) . . . 40
7.7 Summary . . . 44

IV Appendices **46**

8 Appendix: Options and Property Identifiers **47**
8.1 Configuration options . . . 47
8.2 Diagnostic environment variables . . . 47
8.3 Supported CAPE-OPEN property identifiers . . . 48

9 Bibliography **50**

Bibliography **51**

List of Figures

2.1 The compound-selection / EOS / PH/PS-strategy configuration dialog shown by `Edit()`. See *Design Decisions* for the PH/PS strategy control this dialog exposes. 9

2.2 COFE's "Select Package or Package Manager" dialog, showing the CoolProp Property Package (and the companion CoolProp Incompressible package) registered alongside GERG-2008, ThermoCalc, ChemSep, TEA, and the other CAPE-OPEN 1.1 packages installed on the system. 10

3.1 The PH/PS flash strategy control, alongside the EOS selector, in the `Edit()` configuration dialog (shown here defaulted to the C# strategy). 17

6.1 The incompressible-fluid selection / concentration dialog shown by `Edit()` on the companion package, here configuring MIT Seawater ("Sharqawy2010" per CoolProp's own origin/reference tag) at 6% mass concentration. The displayed freezing point, `0.00 K (-273.1 °C)`, is the no-correlation-available case explained above, not a real result. . . 29

7.1 The CHP steam-Rankine cycle flowsheet in COFE (original flowsheet by Torbjørn Pettersen, Thermos AS, 2009), with the stream table and turbine/boiler/client duty summary as solved. 32

7.2 Turbine shaft power, boiler/CHP-heat/condenser duty, and overall efficiency: COCO's IAPWS-IF97 water package vs CoolProp (HEOS, IAPWS-95). 32

7.3 The TEALARC dual mixed-refrigerant LNG liquefaction flowsheet in COFE. 33

7.4 Compressor shaft power by machine, the C2/C3 condenser P1 vapour fraction, and LNG recovery: native TEA/SRK vs CoolProp (HEOS), fixed compound-order alignment. . . . 34

7.5 The CO2-from-NG extractive-distillation flowsheet (IECR 52, p.10780) in COFE. 35

7.6 Column duties, product purity, cross-contamination, and recovery: native ChemSep/PPR78 vs CoolProp (HEOS). 36

7.7 The Snøhvit single mixed-refrigerant LNG liquefaction flowsheet in COFE. 37

7.8 MR compression-train shaft power by machine and net train power: native ChemSep/PR76 vs CoolProp (HEOS). 38

7.9 The CScasebook cryogenic air-separation flowsheet in COFE (ChemSep case study): three-stage main air compression, turbo-expander, high- and low-pressure columns (HPC/LPC) and the crude-argon column (ARC), producing gaseous oxygen, gaseous nitrogen and liquid argon. 39

7.10 Main air-compression train shaft power by machine, net ASU power and specific power per tonne of oxygen product: native ChemSep/PR76 vs CoolProp (HEOS). 39

7.11 COFE flowsheet of the CO_2 compression, dehydration, purification and liquefaction (CCS) process: multi-stage integrally-geared CO_2 compression with interstage cooling and knock-out, a direct-contact cooler, desiccant dehydration, distillation-based non-condensable-gas removal and liquefaction, with a two-stage cascade ammonia refrigeration loop (green). Diagram: `cocosimulator.org/downloads/CO2_compression_liquefaction_Andreasen_2025.png`. 40

7.12 CO_2 compression / dehydration / liquefaction solved in COFE with the native cubic (TEA/SRK) and with CoolProp (HEOS): compressor shaft duty (top left), dehydration reboiler duty (top right), LCO_2 product impurity slip (bottom left; the native N_2 bar is drawn at a floor of 10^{-4} ppm, its labelled value being 2.5×10^{-6} ppm) and CO_2 dissolved in the water condensate (bottom right). 42

List of Tables

7.1 CHP: native IAPWS-IF97 vs. CoolProp IAPWS-95 . . . 33
7.2 TEALARC: native TEA/SRK vs. CoolProp HEOS . . . 34
7.3 CO2-from-NG: native ChemSep/PPR78 vs. CoolProp HEOS . . . 35
7.4 Snøhvit: native ChemSep/PR76 vs. CoolProp HEOS . . . 37
7.5 ASU: native ChemSep/PR76 vs. CoolProp HEOS . . . 40
7.6 CO_2 compression/liquefaction: native cubic vs. CoolProp . . . 41
7.7 Headline agreement across the six validation flowsheets . . . 44

8.1 Single-phase properties (`CalcSinglePhaseProp`) . . . 48
8.2 Two-phase properties (`CalcTwoPhaseProp`) . . . 49
8.3 Compound constants (`GetCompoundConstant`) . . . 49

Implementation, design decisions, flash and property routing, transport-property fallbacks, and validation of the CoolProp-backed CAPE-OPEN 1.1 thermodynamic property package.

Version: 0.4 (draft)
Date: October 2026
Author: Anders Andreasen
Org:

Revision History

Version	Date	Description
0.1	2026-07	Initial draft: architecture, design decisions, flash routing, transport, validation
0.2	2026-07	Air-separation unit (ASU) validation case and PFD
0.3	2026-10	CO_2 compression/liquefaction (CCS) validation case, solved on CoolProp 591e2eff (package 1.1.5)
0.4	2026-10	Review corrections: all six flowsheets re-solved on 591e2eff; actual reference packages and solve modes; pure-fluid PH/PS and fugacity fixes; implementation claims checked against code

Part I

Foundations

Chapter 1

Introduction

1.1 Purpose and scope

This reference documents the **CoolProp CAPE-OPEN property package**: a CAPE-OPEN 1.1 [1] thermodynamic property package that wraps the open-source **CoolProp** library [2] and exposes it to process-modelling environments (PMEs) such as COCO/COFE and ChemSep. The package makes CoolProp's Helmholtz-explicit multiparameter equations of state (the `HEOS` backend — a multi-fluid Helmholtz-energy mixture model in the tradition of [3] and [4], with departure functions in the GERG-2008 style of [5]) and its cubic backends (`PR`, `SRK`) available as a selectable, multi-compound property package with a full vapour–liquid equilibrium flash, single-phase caloric/volumetric/transport properties, and persistence in the flowsheet file.

The package is an **interface layer**, not a thermodynamic engine. Following the CAPE-OPEN adapter pattern, it translates PME requests into calls to CoolProp's low-level `AbstractState` C interface and maps the results back onto CAPE-OPEN property identifiers and SI units (*Architecture*). All of the equation-of-state mathematics — reducing functions, departure functions, density solving, the underlying multiparameter correlations — belongs to CoolProp and is documented in the CoolProp literature [2]; **this reference deliberately does not reproduce it.** What it *does* document is everything that lives in the CAPE-OPEN layer and its thin calculation shim: the design decisions, the flash and property routing, the fallbacks that make the package robust inside a real flowsheet solve, and the validation of that behaviour on six reference process flowsheets.

> **Note**
>
> Throughout, "the package" means the CAPE-OPEN wrapper and its C# calculation shim (`CoolPropCO.CapeOpen` + `CoolPropCO.Core`); "CoolProp" means the upstream native library reached through the `AbstractState` handle.

1.2 What made this possible: the upstream mixture flash

A CAPE-OPEN package is only as usable as its flash is robust. A PME evaluates a property package *thousands* of times per converged flowsheet — at feasible states, at wildly infeasible recycle guesses, at the exact saturation boundary, and on degenerate (single-component, zero-fraction) compositions — and a single unrecoverable failure aborts the whole solve.

For most of CoolProp's history its **mixture** flash was not robust enough for this. The historical `HEOS` mixture `update(PT_INPUTS, ...)` could return a metastable single-phase density root at or near a

phase boundary, fail its successive-substitution / density solve a hair from convergence, or silently converge a state-function (PH/PS) flash to the wrong temperature. Any of these turns a routine flowsheet evaluation into a non-physical result or a hard stop.

Recent upstream improvements to the CoolProp mixture flash are what make this package practical. The more reliable phase-split and stability handling and the improved saturation (`PQ`/`QT`) solvers in recent CoolProp mean that its mixture routines converge reliably enough across the state space that a CAPE-OPEN package can lean on them, with a thin safety net rather than a full re-implementation of the flash. The safety net is not empty: the package still uses its own quality-based state-function flash for mixtures by default (*The C# PH/PS flash*) and its own flash for the pure-fluid cases where CoolProp's native one fails (*PH/PS on a pure feed*).

The package ships a specific CoolProp build, recorded in `coolprop-build.json` next to the bundled `CoolProp.dll` and checked against the DLL at every release: **CoolProp 8.0.1dev, git** `591e2eff`. This is a development build ahead of the latest CoolProp release (v8.0.0): CoolProp master as of 2026-10-01 (at the merge of PR #3272, wide-boiling phase-split recovery) plus PR #3451 (mixture PT-flash speed, merged upstream since) and PR #3473 (systematic liquid-liquid stability probe, open at the time of writing). Statements in this reference about CoolProp's behaviour refer to that build.

The validation flowsheets in *Validation* — a steam cycle, two mixed-refrigerant LNG trains, an extractive distillation, a cryogenic air separation unit and a CO_2 compression and liquefaction train, solved end-to-end in COFE against the PME's own reference packages — demonstrate that maturity in practice.

1.3 What this reference covers

- the **software architecture** and the `AbstractState`-handle model, including the dedicated equilibrium / liquid / vapour handles (*Architecture*);
- the **design decisions** that keep the package robust and correct in a PME: compound-order alignment by CAS, zero-fraction handling, single-component (pure-fluid) routing, and EOS/backend selection (*Design Decisions*);
- the **flash and property routing**: TP, the robust PH/PS strategy and the optional native `HSU_P` path, saturation flashes, and the micro-perturbation convergence retry (*Flash and Property Routing*);
- the **transport-property fallbacks** used when CoolProp has no viscosity/conductivity/surface-tension model for the selected fluid or backend — as is common on the cubic backends (*Transport-Property Fallbacks*);
- the **incompressible-fluid** companion package for utility streams (*The Incompressible Companion Package*);
- **validation** of the package on six flowsheets in COFE (CHP, TEALARC, CO2-from-NG, Snøhvit, ASU, and CO_2 compression/liquefaction) against the PME's own reference packages (*Validation*);
- an **options and identifiers** reference for configuration and property support (*Appendix: Options and Property Identifiers*).

1.4 Audience

The reader is assumed to be familiar with process thermodynamics, the vapour –liquid equilibrium flash, and the CAPE-OPEN 1.1 thermodynamic interfaces. The CoolProp equations of state themselves are treated as a trusted dependency.

Chapter 2

Architecture

2.1 The adapter pattern

The package follows the standard CAPE-OPEN adapter pattern [1]: the CAPE-OPEN COM layer contains **no thermodynamic calculation logic**. It reads T/P/composition from the PME's Material Object, calls into CoolProp for every actual EOS/flash/transport calculation, and writes the results back onto the Material Object in CAPE-OPEN's units and property identifiers.

```
PME (COFE / ChemSep)
  |
  v
Material Object                          (implemented by the PME)
  |  ICapeThermoMaterial, ICapeThermoPhases, ICapeThermoCompounds
  v
CoolPropCO.CapeOpen                      (the interface layer - this package)
  |  ICapeThermoPropertyRoutine, ICapeThermoEquilibriumRoutine,
  |  ICapeThermoCompounds, ICapeThermoPhases, IPersistStreamInit, ...
  v
CoolPropCO.Core                          (thin calculation shim)
  |  ICoolPropEngine: Configure / Flash / CalcSinglePhaseProps /␣
↪SurfaceTension
  v
CoolProp (native)                        (AbstractState C interface, P/Invoke)
```

Three projects implement the shipped package:

Project	Responsibility
`CoolPropCO.CapeOpen`	The COM-visible property package (net48, x64). Implements every CAPE-OPEN 1.1 interface, the `MaterialObjectWrapper` (safe VARIANT coercion), persistence, logging, and COM registration.
`CoolPropCO.Core`	Compound catalog, models (`Component`, `FlashSpec`, `FlashResult`, `EosModel`, `PhPsStrategy`), the `ICoolPropEngine` seam, and the real P/Invoke engine (`CoolPropEngine`) plus a non-calculating `StubEngine` used before the engine is wired or for interface-only testing.
`CoolPropCO.Editor`	The Avalonia 11 configuration GUI shown by `Edit()` (compound selection, EOS, PH/PS strategy, kij overrides), and the companion incompressible-fluid editor.

A fourth project, `CoolPropCO.CapeOpenNative`, is a hand-written C++ COM port of the same package (same compound handling, flash routing and persistence format, its own CLSID). It is built and tested in CI with its own `*_CoolPropNative.fsd` flowsheets but is not yet part of the portable release; this reference describes the C# package unless stated otherwise.

`ICoolPropEngine` is the single seam between the CAPE-OPEN layer and CoolProp: the CAPE-OPEN code depends only on this interface, never on CoolProp's native API directly. Native interop is flat P/Invoke against CoolProp's integer-handle `AbstractState_*` C interface — no C++/CLI bridge and no SWIG wrapper, which avoids the mixed-mode COM-activation issues those would introduce inside a PME's process.

2.2 The AbstractState handle model

CoolProp's low-level interface centers on an **AbstractState**: an opaque integer handle representing one EOS/backend/fluid-set combination, updated in place by successive `update(input_pair, v1, v2)` calls. Each update caches the converged state (density, phase) as the density guess for the *next* update on that same handle — this is what makes repeated updates on a warm handle fast, but it also means **a handle's history can bias a later result** if the handle is reused for calls with different intent.

The engine keeps **four kinds of handle**, each with a narrow, fixed purpose, precisely to keep that history from crossing between intents:

Handle	Purpose
`_handle`	The **equilibrium** handle. Used for every `CalcEquilibrium` flash (TP, PH, PS, PVF, TVF) and nothing else, including a pure feed in a multi-compound package. Not phase-imposed, except transiently inside two flashes: a TP state on the saturation line (*TP on the saturation line*) and the single-phase branch of the hand-rolled pure PH/PS flash (*PH/PS on a pure feed*). Each imposition covers one update and is released immediately.
`_liqHandle`	Single-phase property calls (`CalcSinglePhaseProp`) for a phase the PME has told us is liquid. Phase-imposed (`specify_phase("phase_liquid")`) for the duration of the call, then unimposed. If no liquid root exists at (T, P), the call falls back as described under *Single-phase property fallbacks*.
`_vapHandle`	The vapour analogue of `_liqHandle`.
`_pure[i]`	One pure-fluid `AbstractState` per configured component, used by **single-phase property calls** when the composition is (numerically) pure in component *i*. Flashes do not use it. See *Single-component (pure) compositions* for why this exists.

Why separate liquid/vapour handles from the equilibrium handle. A single-phase property call *imposes* a phase on its handle (`specify_phase`) so CoolProp's density solver doesn't have to rediscover which root is wanted. If that same handle were later reused for an *unimposed* equilibrium flash, the leftover imposed-phase state and density guess can bias `update(PT_INPUTS, ...)` onto the wrong (sometimes **metastable**) root for a state that is genuinely two-phase. This was observed directly during validation on the TEALARC flowsheet's C2/C3 condenser: a state that is unambiguously two-phase ($Q \approx 0.4$) in isolation came back as all-liquid inside the live flowsheet solve. Giving the equilibrium flash its own handle — one that is never touched by a single-phase property call, and whose only phase impositions are the single-update ones noted above — removes that specific cross- contamination path. (For the record: in that particular investigation the actual root cause turned out to be a compound-order mismatch, not handle contamination — see *Design Decisions* — but the handle separation is still the

correct defensive design and is kept regardless, since it eliminates a real class of cross-pollution between the imposed-phase and equilibrium code paths.)

Reconfiguration. `Configure(eos, components, kijOverrides, phPsStrategy)` frees any existing handles and rebuilds all of them — the four equilibrium/ liquid/vapour/pure handles — from the current compound list, applying any kij (binary interaction parameter) overrides keyed by CAS pair. It is called whenever the EOS, compound selection, or compound *order* changes (the last of these matters — see *Design Decisions*), and idempotently on every `Edit()` confirmation and flowsheet `Load()`.

2.3 Pure-fluid routing

CoolProp's **mixture** backend does not automatically route a degenerate (single-component) composition to a pure-fluid solver — it still runs the full mixture machinery, which can fail outright on some pure compositions (observed: a mixture handle throws on certain pure water states that a dedicated pure handle solves without issue; the pure handle was also noticeably faster in development timing, not reproduced here). Since a PME routinely evaluates pure compositions — CAPE-OPEN's reference-state harmonisation pass (§5.6) evaluates every compound individually — single-phase property calls detect an exactly pure composition (one fraction $\geq 1 - 10^{-9}$, all others $\leq 10^{-9}$) and route it to the matching `_pure[i]` handle instead of the mixture handle. Near-pure trace compositions (a small but nonzero secondary component) deliberately stay on the mixture path, so trace tracking through a flowsheet is not silently short-circuited.

Flashes are routed differently. A pure feed is always flashed on the equilibrium handle (with absent compounds floored, *Zero-fraction components*); only its PH/PS flash takes a pure-specific path (*PH/PS on a pure feed*).

2.4 Single-phase property fallbacks

`CalcSinglePhaseProp` asks for one phase's properties at a given $(T, P, \mathbf{x})$. That phase may have no root there (a "liquid" request above the bubble point, a vapour request for a subcooled pure fluid, or a metastable vapour in COFE's reference-state pass). Rather than fail the PME's call, the engine falls back in three steps, **without signalling the caller**:

1. update with the requested phase imposed;
2. if that fails, update without imposition, so the values come from the root CoolProp considers stable, which may be the **other** phase;
3. if that also fails, evaluate at the ideal-gas density $\rho = P/RT$, which returns ideal-gas-state values even for a liquid request.

Separately, if the PME supplies no composition for the requested phase, the package uses the overall composition, and an equimolar one if that is also missing. These fallbacks keep a flowsheet solve alive, but a value obtained through steps 2 or 3 is not a property of the requested phase; treat single-phase properties at unusual states with that in mind. **Implementation**: `CoolPropEngine.CalcSinglePhaseProps()`, `CoolPropPropertyPackage.CalcSinglePhaseProp()`.

2.5 COM surface and lifetime

The package implements the full CAPE-OPEN 1.1 interface set required of a property package (*Introduction*): `ICapeThermoPropertyRoutine`, `ICapeThermoEquilibriumRoutine`, `ICapeThermoMaterialContext`, `ICapeThermoCompounds`, `ICapeThermoPhases`, `ICapeThermoUniversalConstant`, `ICapeIdentification`, `ICapeUtilities`, and

`IPersistStreamInit` for flowsheet persistence. COM lifetime follows the pattern the CAPE-OPEN skill's field notes document as correct: the only external COM references the package holds — the Material Object and the simulation context — are released via `Marshal.ReleaseComObject` in `UnsetMaterial`/`SetMaterial` (on replacement) and `Terminate`, with a residual-reference-count sentinel logged if a release is ever incomplete. The `IStream` handed to `Save`/`Load` is the PME's, not ours, so it is wrapped with a no-op `Dispose`.

Two GUIDs identify the package: the Property Package CLSID (`E5A65162-CFB9-4394-8AAC-FAEA7239E420`, ProgId `CoolPropCO.PropertyPackage`) and the Package Manager CLSID (`FE29F896-33CA-4F0D-9147-2AFB91B7550D`, ProgId `CoolPropCO.PackageManager`); the companion incompressible package (*The Incompressible Companion Package*) has its own CLSID (`837C291D-A3C5-4E4E-A981-448A6F0896FA`, ProgId `CoolPropCO.IncompressiblePackage`). Both property packages register in the CAPE-OPEN Property Package category (`CF51E384-0110-4ED8-ACB7-B50CFDE6908E`) and the manager in the Property Package Manager category (`CF51E383-0110-4ED8-ACB7-B50CFDE6908E`). The default registration (`register.bat`) is per-user (HKCU) and needs no administrator rights; `register-machine.bat` registers machine-wide (HKLM), which headless solvers and elevated or service contexts need, because they do not see another user's HKCU registration.

2.6 Configuration GUI

`Edit()` runs the Avalonia configuration dialog on a dedicated, long-lived STA thread separate from the PME's own COM STA, and blocks the calling thread until the dialog closes. Two robustness measures matter for a COM-embedded GUI specifically:

- **Software rendering is forced.** GPU/ANGLE/OpenGL initialisation can hang indefinitely on remote-desktop sessions, VMs, and machines without GPU drivers — none of which a configuration dialog needs.
- **The UI-thread startup wait is bounded** (30 s). If the UI stack fails or stalls, `Edit()` logs the failure and returns gracefully instead of blocking the PME forever.

2.7 Registered as a selectable property package

Once COM-registered, the package appears in the PME's own package chooser alongside every other CAPE-OPEN 1.1 package on the system — evidence that COM registration and the CAPE-OPEN category GUIDs (above) are correctly wired end-to-end, not just that the package builds.

2.8 Persistence

Configuration is saved into the flowsheet file via `IPersistStreamInit` as a small versioned binary stream: format version, EOS selection, the ordered compound list (CoolProp name + CAS per compound), kij overrides (keyed `"CAS1|CAS2"`), and the PH/PS strategy. The stream version lets the format grow — a v1 stream (saved before the PH/PS strategy field existed) loads cleanly with that field defaulting to the C# strategy. Because COFE creates a fresh property-package COM instance per solve cycle, the *last-edited* configuration is also broadcast through static fields shared across instances within one PME session, so instances created mid-solve (without their own `Load()`) pick up the same compounds/EOS/strategy as the instance the user configured via `Edit()`.

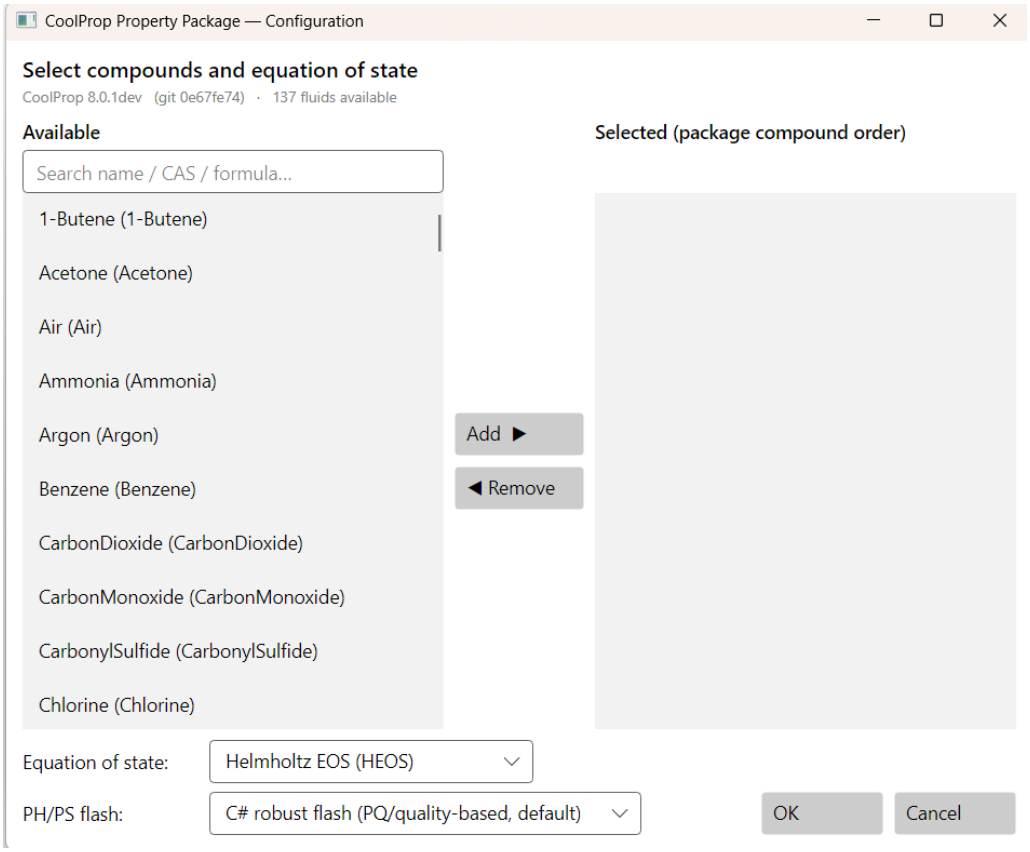


Figure 2.1: The compound-selection / EOS / PH/PS-strategy configuration dialog shown by `Edit()`. See *Design Decisions* for the PH/PS strategy control this dialog exposes.

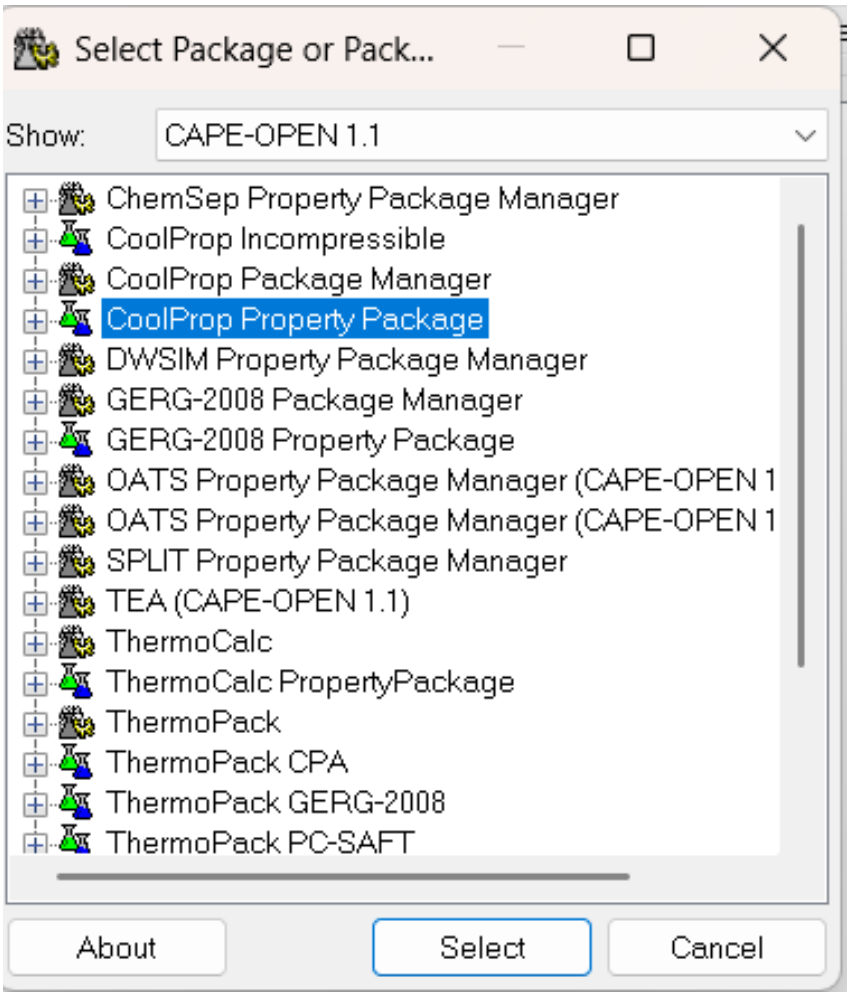


Figure 2.2: COFE's "Select Package or Package Manager" dialog, showing the CoolProp Property Package (and the companion CoolProp Incompressible package) registered alongside GERG-2008, ThermoCalc, ChemSep, TEA, and the other CAPE-OPEN 1.1 packages installed on the system.

Warning

Those static fields are shared by **every** CoolProp package instance in the process. With two CoolProp packages in one flowsheet configured differently (for example HEOS on one material type and PR with kij overrides on another), an instance that has not been loaded inherits whichever configuration was saved or edited last. CAS realignment at the first calculation (*Compound-order alignment to the Material Object*) corrects its compound list but not its EOS, kij overrides or PH/PS strategy. All six validation flowsheets use a single CoolProp package, so they are unaffected. The C++ port does not use shared statics and relies on per-instance `Load()` and CAS realignment alone.

Part II

Implementation and Design Decisions

Chapter 3

Design Decisions

This chapter documents the decisions in the interface layer that are not obvious from the CAPE-OPEN spec alone: how zero/near-zero compositions are handled, how single-component ("pure") states are routed, how compound order is kept consistent with the PME, how the EOS/backend is selected, and how the PH/PS flash strategy is chosen. Each of these was driven by a concrete flowsheet failure mode, not a hypothetical one — the failure is described alongside the fix.

3.1 Zero-fraction components

A property package is normally configured with more compounds than any one stream actually uses — a 5-compound refrigerant package serving a 2-compound refrigerant loop, for example. CAPE-OPEN composition arrays are dense (one entry per configured compound, in a fixed order — see *Compound-order alignment to the Material Object*), so the unused compounds arrive as **exact zeros**, not as absent entries.

CoolProp's `HEOS` mixture reducing and departure functions divide by composition-weighted terms. An exact-zero component makes those terms singular, and the density solver returns a non-physical result (`solver_rho_Tp ... p is not a valid number`) — for *every* flash on that stream, at *any* T/P, not just at edge cases. This was root-caused on the TEALARC flowsheet's methane/ethane refrigerant loop, configured against a 5-compound package: a feed of `[0.6, 0.4, 0, 0, 0]` fails outright, while flooring the three zeros to 10^{-12}–10^{-8} and renormalizing succeeds and reproduces a true 2-compound package's result to 10 significant figures.

The fix, applied uniformly wherever a composition is handed to CoolProp (`Flash`, `CalcSinglePhaseProps`, `SurfaceTension`): any mole fraction below a floor of 10^{-10} is lifted to 10^{-10} and the vector is renormalized before `set_fractions`.

$$x_i \leftarrow \max(x_i, 10^{-10}), \qquad x_i \leftarrow x_i \Big/ \sum_j x_j \tag{3.1}$$

10^{-10} is small enough to be thermodynamically negligible (many orders of magnitude below any flash convergence tolerance) while keeping the mixture's reducing functions well-conditioned. **Implementation**: `CoolPropEngine.SetFractionsSafe()`, `CompFloor = 1e-10`.

> **Note**
>
> This is distinct from the *exactly*-pure case below. For single-phase property calls an exactly pure composition is routed to a different (pure-fluid) handle entirely, not clamped. Flashes of a pure feed do use this clamp on the mixture handle (*PH/PS on a pure feed*).

3.2 Single-component (pure) compositions

The full mechanism and rationale are described in *The AbstractState handle model* (*Architecture*) alongside the rest of the handle model; summarized here as a design decision: CoolProp's **mixture** `AbstractState` backend does not itself detect and short-circuit to a pure-fluid solver when handed a degenerate one-component composition, and can fail outright on pure compositions the dedicated pure-fluid solver handles without issue (observed on pure water at the reference-state conditions COFE evaluates on package import). Since a PME evaluates pure compositions routinely — not only when a stream happens to be single-component, but as part of CAPE-OPEN's mandatory reference-state pass over every configured compound — the engine maintains one pure `AbstractState` per configured compound (`_pure[i]`) built at `Configure()` time, and routes the **single-phase property calls** of an **exactly** pure composition ($x_i \geq 1 - 10^{-9}$, all others $\leq 10^{-9}$) to that handle instead of the mixture handle. Equilibrium flashes are not routed this way: they run on the equilibrium handle with absent compounds floored, and a pure feed's PH/PS flash has its own path (*PH/PS on a pure feed*).

The exact-purity gate is deliberate: a composition that is *nearly* pure but carries a genuine trace of a second component stays on the mixture path, so trace-component tracking through a flowsheet (e.g. a ppm-level contaminant) is never silently dropped by snapping to the pure solver.

Pure-handle routing is also faster: CoolProp's dedicated pure-fluid saturation and density solvers were noticeably quicker than the mixture backend at the same degenerate composition in development timing (not reproduced here). That matters because the reference-state pass touches every compound on every package instantiation.

3.3 Compound-order alignment to the Material Object

CAPE-OPEN composition arrays are **positional**: element i is "the i-th compound," with no identity carried alongside it. The order is established once, by whichever side of the interface the PME trusts as ground truth for a given material — and that order is **not guaranteed to match the property package's own saved compound order**. A flowsheet's compound list can be re-ordered by the PME's own bookkeeping, restored from a saved file whose order predates a later `Edit()`, or simply differ from whatever order the package happened to build its selection in.

If the two orders disagree, the feed **silently scrambles**: the mole fraction the PME intends for compound A lands on the package's compound B, and the flash returns a plausible-looking but wrong result — not an error. This was root-caused as the underlying cause of an extended investigation into why the TEALARC and Snøhvit flowsheets' CoolProp-backed results differed materially from the native cubic reference (*TEALARC: dual mixed refrigerant, the C2/C3 case study*, *Snøhvit: single mixed refrigerant, multi-stage compression*): the package had persisted its compound selection as `[Methane, Ethane, n-Butane, n-Propane, Nitrogen]` while the flowsheet's own order was `[Methane, Ethane, Propane, N-butane, Nitrogen]` — propane and n-butane transposed. Propane's mole fraction was therefore flashed as n-butane throughout the flowsheet; at the C2/C3 refrigerant condenser (50/50, 30 °C, 22.5 bar) that turns a genuinely two-phase state ($Q \approx 0.40$ for ethane/propane) into an all-liquid one (ethane/n-butane *is* liquid there), which cascaded into heat-exchanger temperature-limit warnings throughout the downstream cold end.

The fix follows directly from the CAPE-OPEN 1.1 specification (§7 — `ICapeThermoCompounds`) and is unambiguous once stated: the Material Object is the PME's own authority on a stream's compound order, so the package should adopt it rather than assume agreement. Before every positional composition read — in both `CalcEquilibrium` and `CalcSinglePhaseProp` — the package now:

1. reads the material's own compound list via `material as ICapeThermoCompounds -> GetCompoundList()` (`MaterialObjectWrapper.GetCompoundOrder()`);

2. resolves each entry to the package's catalog **by CAS number** (the stable identity key — see below);
3. if every compound resolves and the order differs from the package's current selection, adopts the material's order as authoritative (`AlignSelectionToMaterial`), and the engine is reconfigured (handles rebuilt) the next time it is used.

This makes the package's own internal/persisted order irrelevant to correctness — it is realigned to the PME's ground truth before it can ever matter — and it is a no-op (zero reconfiguration cost) whenever the two orders already agree, which is the common case. It also degrades gracefully: if the material does not expose `ICapeThermoCompounds`, or any compound fails to resolve, the package falls back to its existing positional order rather than risk an incorrect realignment.

Why CAS, not name. The CAS registry number is the most stable identifier CAPE-OPEN exposes for comparing compounds across packages and PMEs (the `casRegistryNumber` compound constant; optional in the standard, but populated by the PMEs used here). Display names and formulae are not stable (a package's catalog and the PME's compound database can spell the same compound differently — "n-Butane" vs. "N-butane" is exactly the pair that triggered this investigation). The package's compound catalog carries a verified CAS number for every fluid it exposes (sourced from CoolProp's own `get_fluid_param_string(fluid, "CAS")`), so CAS-keyed resolution is reliable in both directions.

3.4 Equation-of-state / backend selection

One property package instance, one EOS at a time, selectable in the `Edit()` GUI and persisted with the flowsheet (not separate packages per EOS — a flowsheet swaps the EOS by editing the one package, not by rewiring which package a stream uses).

Selection	CoolProp backend	Notes
Peng-Robinson	`PR`	Cubic. No native transport model — see *Transport-Property Fallbacks* for the fallback correlations this triggers.
Soave-Redlich-Kwong	`SRK`	Cubic. Same transport-fallback behaviour as PR.
Helmholtz EOS	`HEOS`	CoolProp's multi-fluid Helmholtz-energy mixture model (*Introduction*). Highest thermodynamic accuracy; the default. Native transport models exist for most pure fluids; for mixtures CoolProp uses a simple mole-fraction mixing rule (*Transport-Property Fallbacks*).

Only the **HEOS** backend is validated in *Validation*, against the PME's own reference packages (cubic and IAPWS-IF97). The CoolProp `PR` and `SRK` backends are not validated there.

> **Note**
>
> On the cubic backends, CoolProp's native state-function flash cannot be used for a pure fluid (it throws `type not set`, up to at least `591e2eff`; a fix is expected in CoolProp 8.1). The package handles pure PH/PS on PR/SRK with its own flash (*PH/PS on a pure feed*). Genuine mixtures are not affected.

3.4.1 Cold-start initialization on hard mixtures

HEOS's mixture density solver is, in general, more accurate but less uniformly robust across a wide composition/temperature range than a cubic EOS — the cubics were built for exactly this kind of numerical

robustness across a broad flash-initialization search, at some cost in accuracy near difficult regions (e.g. an azeotrope). This shows up concretely in a PME's **initial-guess search**: when a flowsheet is solved from a fully cold start (no saved converged state to warm-start from — e.g. after a manual Reset), the PME's search can probe extreme trial compositions a converged solve never visits, and HEOS's mixture solver can fail outright on one of those trial points even when the flowsheet's actual converged solution is well within HEOS's normal operating range. This was observed directly on the CO2-from-NG extractive-distillation flowsheet in *CO2-from-NG: extractive distillation, a hard mixture and a cold-start caveat*: solved cold, HEOS's density solver fails at an extreme trial point during the initial-guess search; solved from the flowsheet's saved state (or any warm start), it converges cleanly with no errors — the same physical problem, the same converged answer, the only difference is whether the solver ever has to evaluate that one extreme point.

Practical guidance, confirmed on this flowsheet: for a wide-boiling-range or otherwise numerically hard mixture (light components alongside a much heavier one, near an azeotrope, or any flowsheet that fails only on a cold/Reset solve), do an initial solve with the **same** CoolProp property package set to a **cubic backend** (PR or SRK — fast and robust across a wide composition range) to get the flowsheet to a converged state, then, on that CoolProp package instance, switch the EOS selection to **HEOS** and re-solve from the now warm-started state. Since the EOS selection is a single `Edit()` setting on one package instance (*Equation-of-state / backend selection*), no flowsheet restructuring or second property package is needed — reset -> solve under CoolProp/cubic -> switch to CoolProp/HEOS -> solve. Verified end-to-end on *CO2-from-NG: extractive distillation, a hard mixture and a cold-start caveat*: this exact sequence converges cleanly where a cold HEOS solve does not.

3.5 PH/PS flash strategy

The state-function (PH/PS) flash — find the state matching a target enthalpy or entropy at fixed pressure — is the one flash CoolProp does not solve as a single robust native call across the package's whole operating history (see *Flash and Property Routing* for the detailed mechanics and the historical native-flash failure mode this motivated). The package therefore offers **two interchangeable strategies**, selectable in the `Edit()` GUI next to the EOS dropdown and persisted with the flowsheet:

Strategy	Behaviour
C# (default)	A quality-based flash: bracket the target between the bubble and dew properties via CoolProp's saturation (`PQ`) flash, and if the target lies inside that bracket, invert the smooth, metastable-free $H(Q)$/$S(Q)$ relation directly on quality; otherwise solve on temperature. The achieved property is checked against the target, but a miss is only traced, not rejected (*Flash and Property Routing*). This is the strategy used by five of the six validation flowsheets (*Validation*).
Native	CoolProp's own mixture `HSU_P` (PH) / `PSmolar` (PS) flash, checked against the target with the same tolerance — falling back to the C# strategy, per flash, if the native call throws or the check fails. With the compound-order fix in place (above), the native flash converged without fallbacks on TEALARC in development runs, a material change from earlier CoolProp releases where it would occasionally converge silently to the wrong temperature. It can still converge to a measurably different result at a marginal (near-zero-approach) operating point than the C# strategy does. The CO_2 compression/liquefaction validation flowsheet uses this strategy (*Validation*).

The strategy applies to mixtures. A pure feed's PH/PS flash ignores it (*PH/PS on a pure feed*).

Both strategies are always available without a rebuild — the choice is a configuration setting,

not a compile-time one — so a flowsheet that behaves better under one strategy can switch without code changes, and the setting travels with the flowsheet file. An environment variable (`COOLPROPCO_PHPS=native` / `=csharp`) can override the configured strategy for diagnostics (e.g. CI runs, A/B benchmarking) without touching the saved flowsheet state.

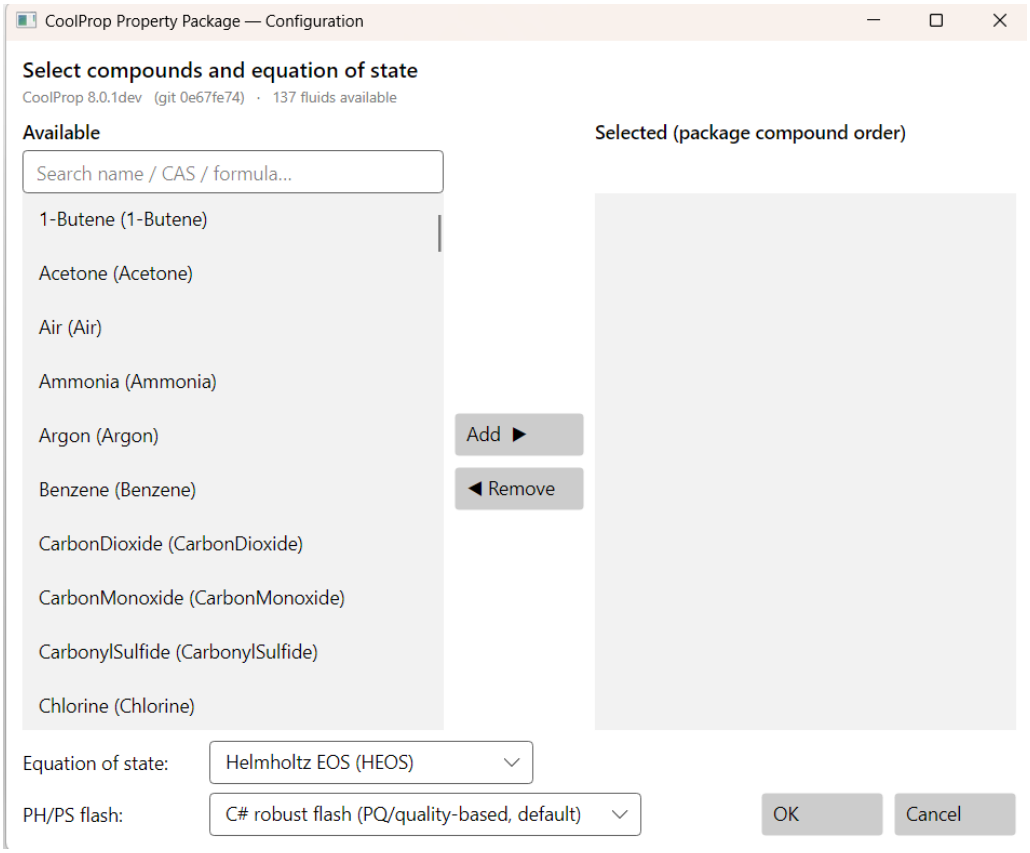


Figure 3.1: The PH/PS flash strategy control, alongside the EOS selector, in the `Edit()` configuration dialog (shown here defaulted to the C# strategy).

Chapter 4

Flash and Property Routing

4.1 Flash routing overview

`CalcEquilibrium` dispatches on the pair of specifications the PME supplies (*Introduction*, CO 1.1 §7). Three of the five flash types (TP, PVF, TVF) route directly to a native CoolProp input pair on the equilibrium handle (*The AbstractState handle model*), TP with a saturation-line fallback; the state-function flashes (PH, PS) are handled by dedicated logic described below. All flashes run on the equilibrium handle, including a pure-compound feed in a multi-compound package (the pure-fluid handles of *Single-component (pure) compositions* serve single-phase property calls only).

Flash	CoolProp input pair	Notes
TP	`PT_INPUTS`	Direct. Wrapped in the micro-perturbation retry (below), with a saturation-line fallback (*TP on the saturation line*).
PVF	`PQ_INPUTS`	Bubble/dew (saturation) flash at fixed pressure. Direct; also perturbation-wrapped.
TVF	`QT_INPUTS`	Bubble/dew flash at fixed temperature. Direct; also perturbation-wrapped.
PH	see below	Mixture: *The C# PH/PS flash* (default) or *The native PH/PS flash (opt-in)* (*PH/PS flash strategy*). Pure feed: *PH/PS on a pure feed*.
PS	see below	Same as PH, on entropy.

> **Note**
>
> **Two phases at most, labelled vapour and liquid.** The package reports at most two phases, and a two-phase result is always labelled Vapour and Liquid. The bundled CoolProp build includes a liquid-liquid stability probe (PR #3473), so on a system with a genuine liquid-liquid split the flash can return two **liquid** phases, which the package would still label Vapour and Liquid. A second liquid phase (VLLE) is outside the package's scope; flowsheets where one can form should be checked for it.

4.2 TP on the saturation line

CoolProp's HEOS PT flash refuses a pure-fluid state within 10^{-4} % of the saturation pressure: it will not choose a density root there and throws instead. Cubic packages and the IAPWS-IF97 water package classify such a state and return a saturated single-phase root, so the package does the same. On that specific failure:

1. If the PME supplied the stream's overall molar enthalpy, the package evaluates the bubble and dew enthalpies at P and forms $Q = (H - H_{\text{liq}})/(H_{\text{vap}} - H_{\text{liq}})$. A value strictly inside $(0, 1)$ is returned as a two-phase `PQ` state; otherwise the liquid ($Q \leq 0.5$) or vapour root is selected.
2. Without an enthalpy hint, the phase is classified by comparing P with $P_{\text{sat}}(T)$ ($P > P_{\text{sat}}$ gives liquid, otherwise vapour).

The chosen phase is imposed on the equilibrium handle for a single PT update and released immediately, so the imposition does not persist into later flashes.

Implementation: `CoolPropEngine.FlashTp()`, `ImposePhaseAtTp()`.

4.3 The C# PH/PS flash

CoolProp's `update(PT_INPUTS, ...)` can, at or near a phase boundary, return a **metastable** single-phase density root instead of the stable two-phase result — verified directly: evaluated exactly at the bubble or dew temperature, it can report a single-phase quality ($Q = -1$) rather than the physical boundary value. Bisecting a state-function flash on temperature using such calls as the residual is fragile exactly where it matters most: the bracket endpoints can read single-phase enthalpies for a target that is genuinely two-phase, and the solve clamps to a temperature far from the true answer (observed: a 95% enthalpy error on a mixed-refrigerant loop during early validation).

CoolProp's **saturation** (`PQ`) flash does not have this failure mode, and enthalpy/entropy are smooth, monotone functions of quality along the saturation line at fixed pressure. The C# strategy uses that fact directly:

1. Evaluate the bubble ($Q = 0$) and dew ($Q = 1$) property at the flash pressure via the saturation flash.
2. **If the target lies inside** $[\,\text{bubble},\ \text{dew}\,]$, the state is two-phase: invert the monotone $H(Q)$ / $S(Q)$ relation directly on $Q \in [0, 1]$ using the Illinois method (never touches the metastable-prone unimposed PT path).
3. **Otherwise** the state is genuinely single-phase: solve on temperature via *Blind temperature solve (single-phase branch)*, then commit that state with a final PT update (wrapped in the perturbation retry, below).

4. **Check**: read back the achieved property and compare against the target ($|\text{got} - \text{target}| \leq 10^{-2} + 10^{-3}|\text{target}|$). A miss is traced as `PHSMISS` (non-fatal — failing hard here would abort the PME's solve iteration on what is typically a transient, unreachable recycle guess rather than a real infeasibility) and the flash still returns its best-effort result. The check is diagnostic only, and the trace line is written only at info level (`COOLPROPCO_LOG=info`; *Appendix: Options and Property Identifiers*).

> **Note**
>
> The tolerance has a relative term on a quantity whose zero point is arbitrary (the CoolProp reference state). For $|H| \approx 10^5$ J/mol it admits about 100 J/mol, roughly 3 K for a gas with $C_p \approx 35$ J/(mol K). The same tolerance gates acceptance of the native mixture flash (*The native PH/PS flash (opt-in)*).

Implementation: `CoolPropEngine.FlashPhPs()`.

4.3.1 Blind temperature solve (single-phase branch)

For the genuinely single-phase branch, H and S are monotone increasing in T at fixed P, so the target temperature is found by root-finding the residual $H(T) - H_{\text{target}}$ (or the entropy analogue), each evaluation being one PT flash.

- **Bracket**: start wide (200–600 K) and walk the endpoints inward past any T where CoolProp cannot evaluate, until both residuals are finite; if they don't straddle zero, widen further (up to 10 attempts) before giving up and returning the closer endpoint (a clamp, not a failure — the caller's verification step in *The C# PH/PS flash* will flag it if that clamp is materially wrong).
- **Root-find**: the **Illinois method** (modified regula falsi — a secant step with endpoint-weight halving to prevent the classic stalled-endpoint problem plain regula falsi has on convex functions). This keeps the robustness of a bracketed method while converging super-linearly on the smooth monotone residual, so it needs far fewer evaluations than plain bisection on the same bracket.

Implementation: `CoolPropEngine.BlindSolveT()`.

4.3.2 PH/PS on a pure feed

When the package holds a single compound, or the feed is effectively pure (one mole fraction $\geq 1 - 10^{-9}$), PH and PS ignore the selected mixture strategy and take one fixed path per configuration:

Configuration	Pure-feed PH/PS flash
Single-compound package, HEOS	CoolProp's native state-function flash, with no read-back check and no fallback. A native failure is raised to the PME as an error.
Single-compound package, PR/SRK	Hand-rolled pure flash (below).
Pure feed in a multi-compound package, any EOS	Hand-rolled pure flash (below).

The fixed routing matters. A native/fallback switch gated on a tolerance would make the flash response non-smooth, which breaks the finite-difference Jacobians of recycle and controller solves; each configuration therefore has exactly one path. The hand-rolled flash exists because CoolProp's native one fails in two pure-fluid cases (up to at least `591e2eff`; fixes are expected in CoolProp 8.1): on the cubic backends it throws `type not set` for any pure fluid, and on HEOS a pure feed held on the multi-compound

handle (absent compounds floored, *Zero-fraction components*) does not converge when two-phase or hot supercritical.

Hand-rolled pure flash.

1. Evaluate the bubble and dew property at P with the saturation (`PQ`) flash. For a pure fluid at fixed pressure, H and S are exactly linear in quality, so a target between them gives $Q = (\text{target} - \text{bubble})/(\text{dew} - \text{bubble})$ in closed form, followed by one `PQ` update.
2. Otherwise solve for T on the liquid branch (target below the bubble value, $T \le T_{\text{sat}}$) or the vapour branch (above the dew value, $T \ge T_{\text{sat}}$), with each evaluation a PT update with that phase imposed. Above the critical pressure there is no saturation and no phase is imposed, except where the cubic equation has three real roots and refuses to choose; the gas-like root is then taken above T_c and the liquid-like root below.
3. The root-finder is Newton's method on T with the analytic slope ($\partial H/\partial T|_P = C_p$, $\partial S/\partial T|_P = C_p/T$), safeguarded by a sign-changing bracket and a bisection step whenever Newton would leave it, converged to 10^{-9} relative. A target outside the EOS's temperature range is clamped to the range limit, as the mixture temperature solve does (*Blind temperature solve (single-phase branch)*).

Round-trip tests (PR, SRK and HEOS; ammonia, water and propane; single-compound and floored multi-compound handles; liquid, vapour, two-phase and supercritical states; PH and PS) recover the temperature and vapour fraction exactly.

Implementation: `CoolPropEngine.FlashPhPsPureHandRolled()`, `FlashNativePhPsPure()`.

4.4 The native PH/PS flash (opt-in)

CoolProp also exposes a native mixture state-function flash (`HSU_P` for PH; the entropy analogue for PS). Historically this package did not default to it: earlier CoolProp releases could converge it to a **silently wrong** temperature on hard mixtures, which is worse than an outright failure because nothing signals the result is incorrect. That specific class of unreliability is what the upstream mixture-flash improvements referenced in *Introduction* primarily address, and it is also downstream of the compound-order fix in *Compound-order alignment to the Material Object*: a scrambled feed is itself sufficient to make *any* flash — native or not — converge to a plausible-looking wrong answer.

With both fixed, the native flash is offered as a selectable, **verified** strategy (*PH/PS flash strategy*) rather than a blind default switch:

1. Run the native input-pair update (`HmolarP_INPUTS`/`HmassP_INPUTS` for PH, `PSmolar_INPUTS`/`PSmass_INPUTS` for PS) on the equilibrium handle.
2. Read back the achieved property and compare against the target with the same tolerance as the C# strategy's verification step.
3. If the native call throws, or the check fails, **fall back to the C# strategy for that flash** — the mismatch is traced (`PHNATIVE-FALLBACK`) and the C# path's own result is returned, so a native-flash failure on a mixture never surfaces as a PME-visible error (pure feeds are handled separately; *PH/PS on a pure feed*).

Because the fallback is per-flash and silent to the caller, the native strategy can be selected with no correctness risk beyond "falls back to the proven default" — the only externally visible difference, when the native flash *does* converge, is that it can land on a measurably different temperature than the C# path at a marginal (near-zero-approach) operating point, as noted in *PH/PS flash strategy*.

Implementation: `CoolPropEngine.TryNativePhPs()`.

4.5 Micro-perturbation convergence retry

Independent of which flash type is running, CoolProp's mixture successive- substitution / density solve can fail to converge at a point a hair from a solution — not because the state is infeasible, but because the iteration itself stalls (`"PT flash lost a phase density solve during successive substitution"`, `"solver_rho_Tp ..."`). This is most visible on a PME's recycle-convergence trial compositions, which probe states a real flowsheet solve never lands on for long, and it is history-dependent: the identical $(T, P, \mathbf{x})$ that fails mid-solve can succeed as a fresh, isolated call, because CoolProp's flash carries internal state across the sequence of calls a PME makes.

The retry wraps the TP flash, the PVF/TVF saturation flashes, and the final PT commit of the C# PH/PS flash's single-phase branch. It does **not** wrap the evaluations inside that strategy (the bubble/dew and quality `PQ` calls, the temperature-solve residuals), the native PH/PS paths, or single-phase property calls, which handle failure in their own way. Where it applies, on a recoverable failure (matched by message substring — "density solve", "successive substitution", "solver_rho"), the continuous spec variable is nudged by a small, escalating fraction and the update retried:

$$f \in \{\pm 3 \times 10^{-5},\ \pm 10^{-4},\ \pm 3 \times 10^{-4},\ \pm 10^{-3},\ \pm 3 \times 10^{-3}\}$$

— perturbing P for a `PQ` flash, T otherwise. The first nudge that converges wins. **The returned state is the nudged one**: the flash reports its own T and P to the PME, so the result can miss the specification by up to 0.3% (about 0.9 K at 300 K). Inside a recycle the PME's own iteration usually absorbs this; on a stream that is not iterated nothing corrects it. If no nudge converges, the original exception is rethrown unchanged — a genuinely infeasible specification still fails loudly rather than silently returning a nearby-but-wrong state.

Implementation: `CoolPropEngine.UpdateResilient()`, `IsRecoverableFlashFailure()`, `PerturbFractions`. Recoveries are counted (`PerturbationRecoveries`) and traced through the CAPE-OPEN logger at info level (`COOLPROPCO_LOG=info`) for diagnostics.

Chapter 5

Transport-Property Fallbacks

CoolProp's `HEOS` backend has native, fluid-specific viscosity and thermal conductivity models for most of its pure fluids. For **mixtures** it has no dedicated model: it evaluates each pure fluid's value at the mixture's density and temperature and combines them as $\mu = \exp\left(\sum_i x_i \ln \mu_i\right)$ and $\lambda = \sum_i x_i \lambda_i$, a rule CoolProp itself flags as "highly approximate". Because that call succeeds, the fallback correlations below are **not** used for HEOS mixtures; HEOS mixture transport properties should be treated as estimates. The **cubic** backends (`PR`, `SRK`) do not: they solve density and phase equilibrium from the cubic EOS but carry no transport-property correlations of their own, so a native transport query on a cubic-backend handle simply has nothing to return. A package that only wraps CoolProp's native transport output would leave viscosity and thermal conductivity unavailable to the PME whenever the user selects a cubic EOS.

To keep the property package usable across all three EOS choices, every transport-property request follows a **try-native-first, fall back on failure** strategy:

1. Query CoolProp's native `viscosity` / `conductivity` output on the current `AbstractState` handle.
2. If CoolProp has no model for the fluid/backend combination (the native call fails), fall back to a correlation from `TransportFallback` (`CoolPropCO.Core/Transport/TransportFallback.cs`) — a set of empirical and corresponding-states methods ported from the `ThermoCalc.CapeOpen` reference package's `ViscosityCalc` service.

This means: on `HEOS` the package normally returns CoolProp's own transport values; on `PR`/`SRK` (and on any `HEOS` fluid combination CoolProp itself does not cover) it transparently returns the fallback correlation instead, so the PME always gets a usable number rather than a hard property failure. This chapter documents those fallback correlations, all implemented in `TransportFallback.cs`.

> **Note**
>
> The fallback set here is intentionally smaller than the ThermoCalc reference it was ported from: **water-specific correlations** (DIPPR viscosity/conductivity, IAPWS surface tension, Meybodi hydrocarbon-water interfacial tension), the **Letsou-Stiel** liquid-viscosity alternative, the **Stiel-Thodos high-pressure conductivity correction**, and **Kesler-Lee pseudo-component estimation** are all present in ThermoCalc but not ported here — CoolProp's own fluid database already covers water and the package does not (yet) support user-defined pseudo-components (`Component.IsPseudo` is reserved for a future release). Only the correlations actually needed to cover the cubic-backend gap are implemented.

5.1 Gas Viscosity

The Lee-Gonzalez-Eakin (1966) correlation [6] is used for gas-phase dynamic viscosity. It is a density-corrected exponential model valid for natural-gas-like systems at moderate pressures.

The gas density is computed from the compressibility factor:

$$\rho = \frac{P \cdot 10^5 \, M_w}{Z \cdot 8314 \, T} \cdot 10^{-3} \quad [\text{g/cm}^3] \tag{5.1}$$

where P is pressure [bar], M_w is mixture molecular weight [g/mol], Z is the compressibility factor (from the molar density already computed by CoolProp for the state, so no extra EOS solve is needed), and T is temperature [K]. Temperature converts to Rankine: $T_R = 1.8\,T$.

Three intermediate parameters are computed:

$$\begin{aligned} A &= \frac{(9.4 + 0.02\,M_w)\,T_R^{1.5}}{(209 + 19\,M_w + T_R) \cdot 10^4} \\ B &= 3.5 + \frac{986}{T_R} + 0.01\,M_w \\ C &= 2.4 - 0.2\,B \end{aligned} \tag{5.2}$$

The dynamic viscosity [cP], converted to Pa·s for CAPE-OPEN output:

$$\mu = A \exp(B\,\rho^C) \tag{5.3}$$

Implementation: `TransportFallback.TryViscosity()` -> `LeeGonzalez()`

5.2 Liquid Viscosity

5.2.1 Lohrenz-Bray-Clark (1964)

The LBC method [7] computes liquid viscosity as a residual contribution added to a dilute-gas base, following the formulation in [8] (Eqs. 10.37–10.41). This is the package's only liquid-viscosity fallback (ThermoCalc's Letsou-Stiel alternative is not ported — see the note above).

Dilute-gas viscosity is computed per component using the corresponding-states parameter:

$$\xi_i = \frac{T_{c,i}^{1/6}}{\sqrt{M_{w,i}}\;P_{c,i}^{2/3}} \tag{5.4}$$

where $P_{c,i}$ is in atmospheres. The per-component dilute viscosity depends on reduced temperature $T_r = T/T_{c,i}$:

$$\mu_i^\circ = \begin{cases} 0.00034\,T_r^{0.94}/\xi_i & T_r < 1.5 \\ 0.0001778\,(4.58\,T_r - 1.67)^{5/8}/\xi_i & T_r \geq 1.5 \end{cases} \tag{5.5}$$

The mixture dilute viscosity uses Herning-Zipperer [9] weighting:

$$\mu_{\text{mix}}^\circ = \frac{\sum_i x_i\,\mu_i^\circ \sqrt{M_{w,i}}}{\sum_i x_i \sqrt{M_{w,i}}} \tag{5.6}$$

Residual viscosity is obtained from a fourth-order polynomial in reduced density. The mixture inverse-viscosity parameter is:

$$\psi = \frac{\left(\sum_i x_i T_{c,i}\right)^{1/6}}{\sqrt{\sum_i x_i M_{w,i}}\;\left(\sum_i x_i P_{c,i}\right)^{2/3}} \tag{5.7}$$

The reduced density is $\rho_r = \rho_m/\rho_c$, where the mixture molar density ρ_m is taken directly from CoolProp's converged state (again, no separate EOS evaluation) and $\rho_c = 1/\sum_i x_i V_{c,i}$. The component critical volume $V_{c,i}$ uses CoolProp's reported V_c when available, falling back to the acentric-factor relation $Z_c = 0.2905 - 0.085\,\omega$ otherwise.

The LBC polynomial is:

$$\mathcal{P}(\rho_r) = a_1 + a_2\,\rho_r + a_3\,\rho_r^2 + a_4\,\rho_r^3 + a_5\,\rho_r^4 \tag{5.8}$$

with coefficients:

Coefficient	Value
a_1	0.1023
a_2	0.023364
a_3	0.058533
a_4	-0.040758
a_5	0.0093324

The liquid viscosity [cP]:

$$\mu_L = \frac{\mathcal{P}^4 - 10^{-4}}{\psi} + \mu^\circ_{\text{mix}} \tag{5.9}$$

with a clamp to μ°_{mix} if the total would be negative. (With these coefficients $\mathcal{P}(0)^4 \approx 1.1 \times 10^{-4}$, so in practice the residual term is not negative at low reduced density.) Converted to Pa·s for CAPE-OPEN output.

Implementation: `TransportFallback.TryViscosity()` -> `Lbc()`

5.3 Thermal Conductivity (Gharagheizi 2013)

Both gas- and liquid-phase thermal conductivity use the Gharagheizi [10] quantitative structure-property relationship (QSPR) method — an empirical correlation requiring only T_b, M_w, P_c, and ω per component. Unlike ThermoCalc, no high-pressure (Stiel-Thodos) correction is applied — the fallback is used mainly on the cubic backends (and for any HEOS fluid CoolProp does not cover), where the priority is a physically reasonable low-to-moderate-pressure estimate rather than exact high-pressure behaviour.

Gas mixture conductivity uses the Herning-Zipperer-style $\sqrt{M_w}$ weighting:

$$k_{\text{mix}} = \frac{\sum_i y_i\,k_i\sqrt{M_{w,i}}}{\sum_i y_i\sqrt{M_{w,i}}} \tag{5.10}$$

Liquid mixture conductivity uses simple linear mole-fraction mixing:

$$k_{\text{mix}} = \sum_i x_i\,k_i \tag{5.11}$$

Implementation: `TransportFallback.TryThermalConductivity()` -> `GharagheiziGas()` / `GharagheiziLiquid()`

5.4 Surface Tension

5.4.1 Brock-Bird (1955)

Two-phase surface tension (a `CalcTwoPhaseProp` request, not single-phase; *Appendix: Options and Property Identifiers*) is computed per component using the Brock-Bird [11] corresponding-states method, evaluated on the liquid-phase composition:

$$\begin{aligned} Q &= 0.1196 \left(1 + \frac{T_{b,r} \ln(P_c/1.01325)}{1 - T_{b,r}}\right) - 0.279 \\ \sigma_i &= P_c^{2/3} \, T_c^{1/3} \, Q \, (1 - T_r)^{11/9} \times 10^{-3} \end{aligned} \tag{5.12}$$

where P_c is in bar, $T_{b,r} = T_b/T_c$ is the reduced normal boiling point, and $T_r = T/T_c$. Components at or above their critical temperature (no meaningful surface tension) are skipped. Mixture surface tension uses linear mole-fraction mixing over the liquid composition:

$$\sigma_{\text{mix}} = \sum_i x_i \, \sigma_i$$

The result is in N/m. CoolProp's own surface tension is tried first (see `SurfaceTension()` in `CoolPropEngine.cs`, evaluated on the dedicated liquid handle — *Architecture*), and Brock-Bird is the fallback when it is not available. CoolProp provides surface tension only for pure fluids, so for any mixture the value returned is Brock-Bird.

Implementation: `TransportFallback.TrySurfaceTension()` -> `BrockBirdPure()`

5.5 Component normal boiling point

The Gharagheizi conductivity and Brock-Bird surface-tension correlations need each component's normal boiling point T_b (Lee-Gonzalez-Eakin and LBC do not). When the compound catalog does not supply it directly, it is estimated from the Edmister relation using T_c, P_c, and ω:

$$\begin{aligned} k &= \frac{7}{3} \frac{\omega + 1}{\log_{10}(P_c/P_{\text{atm}})} \\ T_{b,r} &= \frac{k}{1 + k}, \qquad T_b = T_c \, T_{b,r} \end{aligned} \tag{5.13}$$

Implementation: `TransportFallback.Tb()`

Chapter 6

The Incompressible Companion Package

Utility and heat-transfer-fluid streams (glycol/water, brines, thermal oils) do not need — and cannot use — a VLE flash: CoolProp's `INCOMP` backend models them as single-phase liquids fitted directly to density, heat capacity, viscosity, and conductivity correlations, with no equation of state and no vapour phase. Rather than stretch the mixture property package to cover this case, it is a **separate** CAPE-OPEN 1.1 property package (`CoolPropIncompressiblePackage`, CLSID `837C291D-A3C5-4E4E-A981-448A6F0896FA`, ProgId `CoolPropCO.IncompressiblePackage`) purpose-built around what CoolProp's `INCOMP` backend actually provides.

6.1 Scope

One package instance wraps **one** incompressible fluid: either a pure incompressible or one binary solution at a configurable concentration (e.g. propylene glycol/water at a chosen mass fraction). It is liquid-only — one phase, no fugacity, no K-values — reported to the PME as a single pseudo-compound (the fluid itself has no CAS number to report).

CoolProp's incompressible catalogue is loaded from an embedded copy of its fluid-metadata JSON: 126 fluids (74 pure, 52 solutions), each carrying description, literature reference, concentration basis (mass / volume / mole fraction), valid temperature range, and valid concentration range. A solution's concentration is bounded to its fitted range in the `Edit()` GUI (e.g. MEG 0–0.6, aqueous LiBr 0–0.75) — the underlying correlations are not extrapolated outside where they were fitted.

6.2 Why mass basis only

CoolProp's `INCOMP` backend exposes **only mass-basis** properties (density, enthalpy, entropy, heat capacity, viscosity, conductivity); molar quantities — molar mass, molar density/enthalpy/entropy, speed of sound — are not available and error if requested. The package follows this exactly: every property it advertises is on a mass basis.

CAPE-OPEN still expects a package to be able to report a molecular weight (for PME-side mole<->mass bookkeeping on streams that mix compressible and incompressible packages). Since the fluid has no real molar mass, the package reports a **nominal** value (18.015 kg/kmol — water's, chosen only because it's a familiar, unremarkable number). This is safe specifically *because* every property the package reports is mass-basis: the nominal value cancels out of any mole<->mass round-trip the PME performs ($n \cdot h_{\text{molar}} = (m/M_w)(h_{\text{mass}} \cdot M_w) = m \cdot h_{\text{mass}}$) rather than introducing an error.

6.3 Low-level construction

The `AbstractState` is created via `factory("INCOMP", name)`; for a solution, the concentration is set afterward via `set_fractions([x])` — **not** by encoding it into the fluid name string (`"Name[0.25]"`). That bracketed form is accepted by CoolProp's high-level `PropsSI` convenience function but is rejected by the low-level `factory()` call the package uses throughout (*Architecture*), so concentration is always set as a separate step.

6.4 Flashes supported

Only TP, PH, and PS are meaningful for a single incompressible liquid phase (no vapour fraction to specify against):

Flash	CoolProp call	Notes
TP	*(none needed)*	Trivial — T and P are already the two independent variables for an incompressible liquid.
PH	`HmassP_INPUTS`	Mass-basis enthalpy target.
PS	`PSmass_INPUTS`	Mass-basis entropy target.

Requesting PVF/TVF (vapour-fraction) flashes against an incompressible fluid is a modelling error on the flowsheet side, not a state the package attempts to solve — it raises a clear error rather than silently returning a physically meaningless result.

6.5 Freezing point

A solution's freezing point is concentration-dependent, and is exposed as a CoolProp `T_freeze` query on the configured (fluid, concentration) pair — reported in the `Edit()` GUI live as the concentration is adjusted, so a user configuring a glycol mixture can see immediately whether a chosen concentration is adequate for the coldest temperature the stream will see.

> **Note**
>
> Not every fluid in the catalogue has a fitted freezing-point correlation in CoolProp's own data — this is a data-availability gap in CoolProp itself, not a package limitation. For a fluid without one (e.g. MIT Seawater, aqueous LiBr), the query does not raise an error or return NaN: it silently evaluates to 0 K (displayed as `0.00 K (-273.1 °C)` in the GUI — this is what the figure below actually shows for MIT Seawater). **Only a non-zero (non-absolute-zero) freezing point is a real, fitted value** — a reported 0 K means no correlation is available for that fluid, not that it never freezes.

6.6 Configuration GUI

`IncompressibleEditorWindow` lists the catalogue with a search box, a concentration control (a bounded numeric up/down, shown only for solutions and clamped to the fluid's fitted $[x_{\min}, x_{\max}]$ range), and live display of the fluid's description, literature origin, basis, valid temperature range, valid concentration range, and current freezing point. A button opens the fluid's pre-generated fitting-quality report (a PDF hosted on coolprop.org) directly from the dialog, so the correlation's accuracy and validity range can be checked against the source without leaving the GUI.

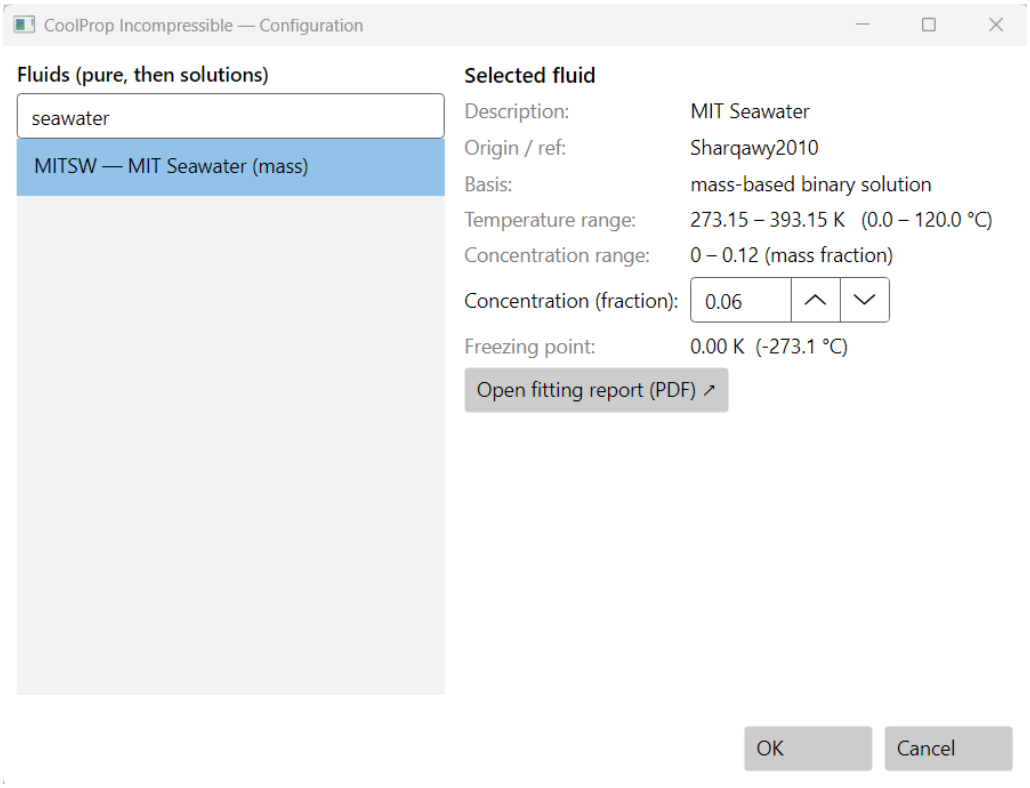


Figure 6.1: The incompressible-fluid selection / concentration dialog shown by `Edit()` on the companion package, here configuring MIT Seawater ("Sharqawy2010" per CoolProp's own origin/reference tag) at 6% mass concentration. The displayed freezing point, `0.00 K (-273.1 °C)`, is the no-correlation-available case explained above, not a real result.

Part III

Validation

Chapter 7

Validation

The package is validated end-to-end, the way it is actually used: six complete process flowsheets, solved in COFE, comparing the CoolProp-backed property package against the PME's own reference package on the *same* flowsheet topology. The reference packages are COCO's IAPWS-IF97 water package (Steamer), COCO TEA with Soave-Redlich-Kwong, and ChemSep with Peng-Robinson 76 or Predictive PR78; each section names the one it uses. This is a deliberately different kind of validation than checking CoolProp's own equations of state against experimental data — that validation is CoolProp's own, upstream (*Introduction*) and is not repeated here. What is validated here is that **the CAPE-OPEN wrapper delivers CoolProp's thermodynamics correctly into a real, converged flowsheet solve**, across single-component, hydrocarbon-mixture, hard-mixture-distillation, and multi-stage-refrigeration cases — which is precisely where an interface bug like the compound-order issue in *Design Decisions* shows up, and an EOS-accuracy check alone would never catch it.

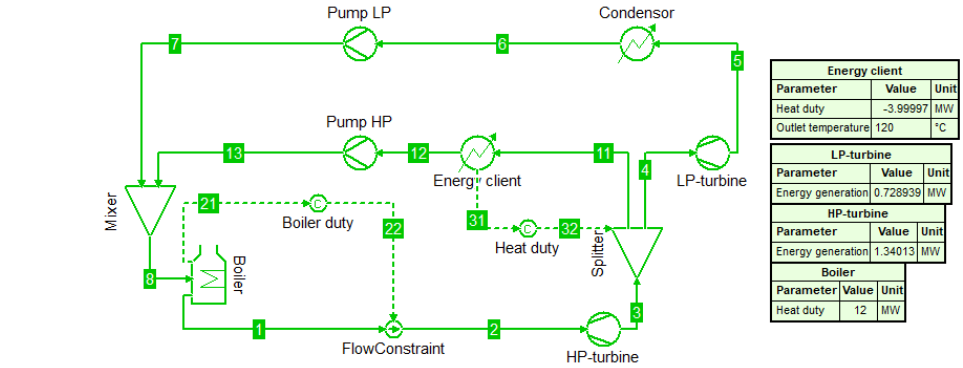
Solve protocol. All twelve files (six native/CoolProp pairs) were re-solved on 2026-10-27 with COFEStand, COCO's headless solver, against the CoolProp build this reference documents (*Introduction*) and the package code it describes, and every solve reported `SOLVE: OK`. Two solve modes were used:

- `COFEStand.exe /r <flowsheet.fsd>` (*reset before solve*) for TEALARC, Snøhvit, ASU and the CoolProp CHP file. The reset does not discard all saved state: TEALARC converges in one iteration and Snøhvit at its initial guess even under `/r`, so for those two the solve confirms the saved state rather than finding it from scratch. ASU (8 and 5 iterations) and CoolProp CHP (6) iterate to convergence.
- Without `/r` (a warm solve from the saved state, as in normal interactive COFE use) for CO2-from-NG and the CO_2 compression train, which do not solve from a reset (*CO2-from-NG: extractive distillation, a hard mixture and a cold-start caveat*, *CO2 compression, dehydration and liquefaction (CCS)*), and for the native CHP file, whose IAPWS-IF97 reference package cannot supply entropy for the wet LP-turbine inlet the reset passes through.

The table in *Summary* lists the mode and the iterations for each file. The comparison scripts read the solved `.fsd` archives directly (no simulation re-run at doc-build time) and are committed alongside their CSV output under `flowsheets/` and `figures/data/` for reproducibility.

7.1 CHP: pure water, single-component routing

A combined-heat-and-power steam-Rankine cycle: a boiler raises steam to 653 K / 22 bar, an HP turbine and LP turbine (extraction, isentropic efficiency specified) generate shaft power, an "Energy client" HeaterCooler extracts process heat at 393 K (the CHP heat offtake), and the condensate is returned to the boiler through two feedwater pumps. Every stream is **pure water** — the exact degenerate composition



Energy client		
Parameter	Value	Unit
Heat duty	-3.99997	MW
Outlet temperature	120	°C

LP-turbine		
Parameter	Value	Unit
Energy generation	0.728939	MW

HP-turbine		
Parameter	Value	Unit
Energy generation	1.34013	MW

Boiler		
Parameter	Value	Unit
Heat duty	12	MW

Stream	1	3	5	6	8	Unit
Pressure	22	2	0.1	0.1	22	bar
Temperature	380	203.294	45.8075	40	72.9334	°C
Flow rate	14.9301	14.9301	8.8635	8.8635	14.9301	ton / h
Enthalpy	13440.7	7619.28	2285.55	-41199.8	-38686.3	J / mol
Mole frac Water	1	1	1	1	1	
Vapor phase						
phase fraction	1	1	0.998936			
Liquid phase						
phase fraction			0.00106402	1	1	

Title:	A simple CHP cycle
Author:	Torbjørn Pettersen
Affiliation:	Thermos AS
Created:	Apr 27, 2009

Figure 7.1: The CHP steam-Rankine cycle flowsheet in COFE (original flowsheet by Torbjørn Pettersen, Thermos AS, 2009), with the stream table and turbine/boiler/client duty summary as solved.

the package routes to a dedicated pure-fluid CoolProp handle rather than the mixture backend (*Single-component (pure) compositions*). This flowsheet is the validation of that routing decision specifically.

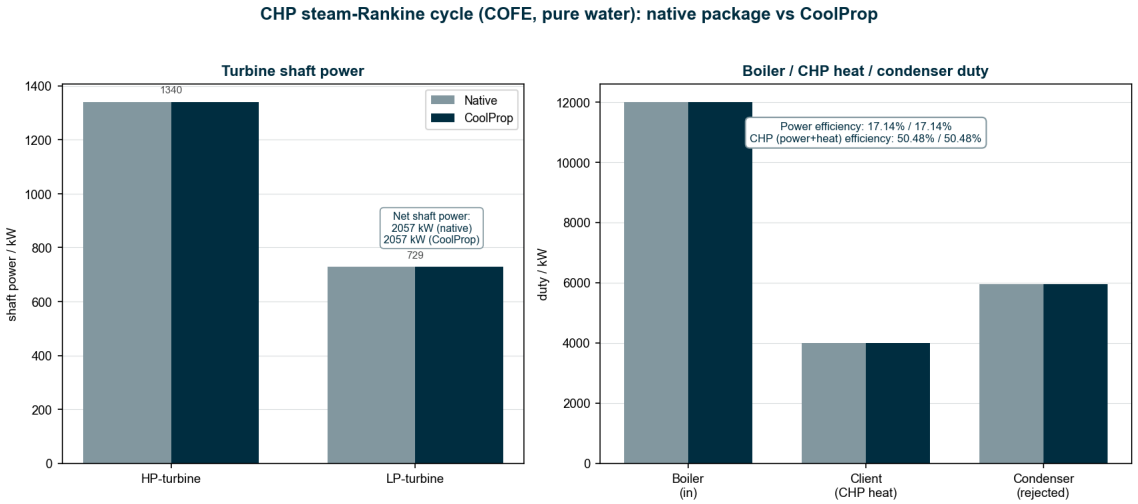


Figure 7.2: Turbine shaft power, boiler/CHP-heat/condenser duty, and overall efficiency: COCO's IAPWS-IF97 water package vs CoolProp (HEOS, IAPWS-95).

Table 7.1: CHP: native IAPWS-IF97 vs. CoolProp IAPWS-95

Metric	Native	CoolProp	Difference
Net shaft power [kW]	2057.1	2057.1	$< 0.01\%$
Boiler duty [kW]	12000.0	12000.0	$< 0.01\%$
CHP heat (client) [kW]	4000.0	4000.0	$< 0.01\%$
Power efficiency	17.142 %	17.143 %	< 0.01 pt
CHP (power+heat) efficiency	50.475 %	50.476 %	< 0.01 pt

Agreement is better than 0.01% on every metric. The two packages evaluate **different** formulations of water: the native reference is COCO's Steamer package, which implements the industrial formulation IAPWS-IF97, and CoolProp evaluates the scientific formulation IAPWS-95 [12]. IF97 is fitted to IAPWS-95, so close agreement is expected, but it is an independent check rather than the same code twice. This is the clean baseline case: no mixture flash and no compound-order question.

7.2 TEALARC: dual mixed refrigerant, the C2/C3 case study

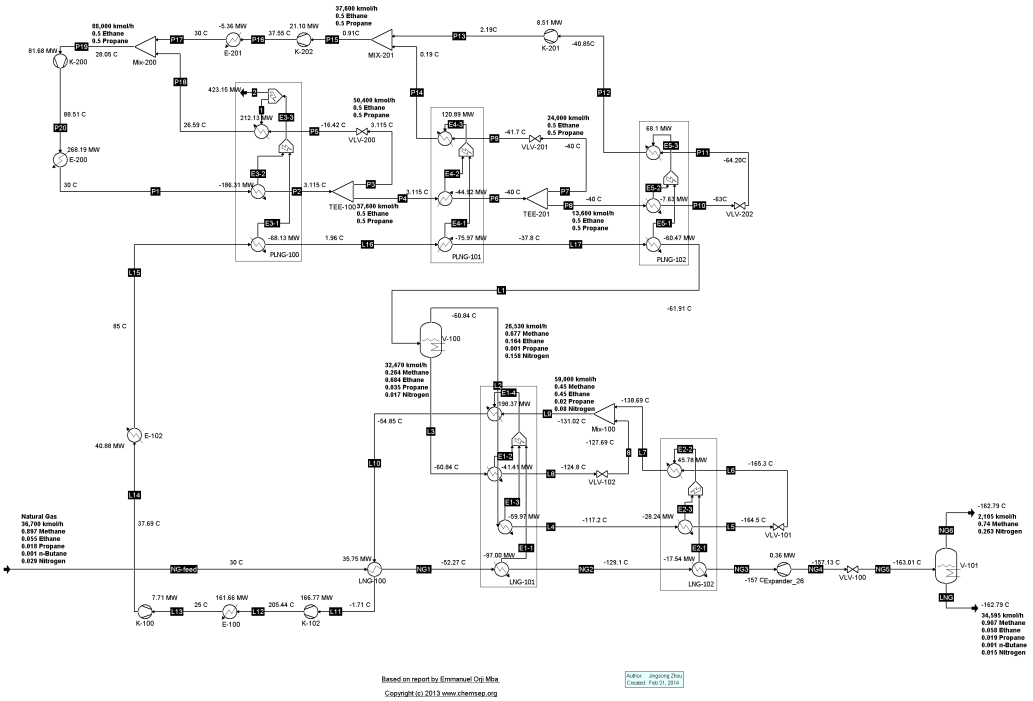


Figure 7.3: The TEALARC dual mixed-refrigerant LNG liquefaction flowsheet in COFE.

A natural gas feed (C1/C2/C3/nC4/N2, 40 bar, 303 K) is liquefied and sub-cooled to LNG (~109 K, 1.1 bar) against two mixed-refrigerant cycles — a 5-compressor train (K-100, K-102, K-200, K-201, K-202), each specifying a pressure ratio and isentropic efficiency rather than a fixed duty, so shaft power is genuinely EOS-computed from the enthalpy rise across each compression stage. This is exactly the flowsheet — and stream P1, the C2/C3 precool loop condenser, is exactly the state — described in *Compound-order alignment to the Material Object*: a 50/50 ethane/propane condenser at 30 °C, 22.5 bar that a compound-order mismatch had previously made read as all-liquid instead of two-phase.

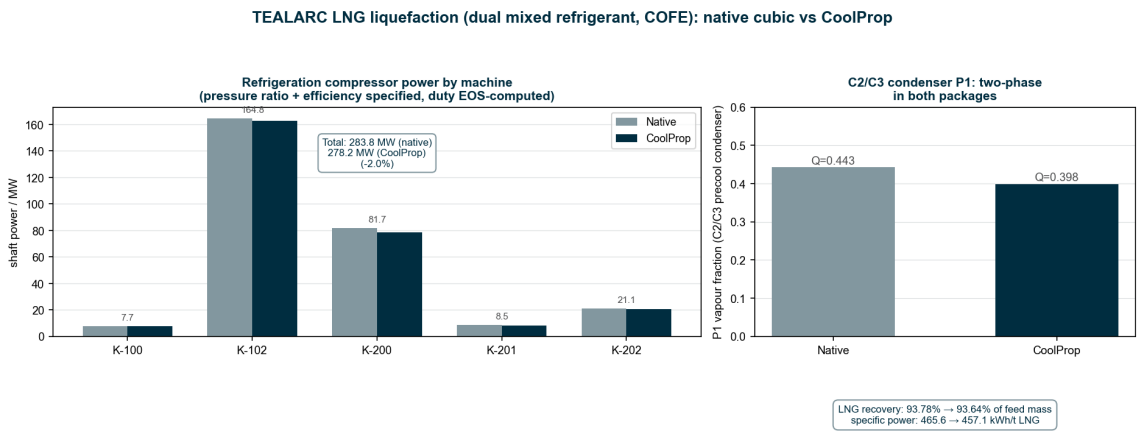


Figure 7.4: Compressor shaft power by machine, the C2/C3 condenser P1 vapour fraction, and LNG recovery: native TEA/SRK vs CoolProp (HEOS), fixed compound-order alignment.

Table 7.2: TEALARC: native TEA/SRK vs. CoolProp HEOS

Metric	Native	CoolProp	Difference
K-100 / K-102 / K-200 [MW]	7.71 / 164.77 / 81.68	7.69 / 162.87 / 78.66	-0.2 / -1.2 / -3.7%
K-201 / K-202 [MW]	8.51 / 21.10	8.30 / 20.69	-2.5 / -2.0%
Total compressor power [MW]	283.78	278.21	-2.0%
P1 vapour fraction Q	0.443	0.398	both two-phase
LNG recovery [% feed mass]	93.78	93.64	-0.14 pt
Specific power [kWh/t LNG]	465.6	457.1	-1.8%

P1's vapour fraction — the specific stream at the center of the compound-order investigation — is genuinely two-phase in **both** packages, with the cubic and HEOS quality differing by an amount consistent with the two EOS's known behaviour on this system, not with a scrambled feed. The 5-compressor power total differs by 2.0%, but the machines do **not** move uniformly: K-100 changes by only 0.2% and K-200 by 3.7%. The spread follows the refrigerant each machine compresses (the two mixed-refrigerant loops have different compositions and suction conditions), so a uniform shift is not expected; what the table rules out is the order-of-magnitude, wrong-phase error the compound-order bug produced.

7.3 CO2-from-NG: extractive distillation, a hard mixture and a cold-start caveat

CO2 and ethane form a low-boiling azeotrope. Column C1 breaks it by extractive distillation with an added heavy (C3–C5) hydrocarbon solvent, producing a CO2 distillate and a bottoms of ethane + solvent; column C2 recovers the ethane overhead and recycles the solvent. The feed spans a much wider boiling range than TEALARC or Snøhvit's refrigerant loops — light CO2 and ethane alongside a C3–C5 solvent — and the CO2/ethane relative volatility near the azeotrope is itself EOS-sensitive, making this a genuinely harder mixture-flash case, run through ChemSep columns rather than simple flash drums.

Warning

Cold-start caveat, and the fix. Solved from a fully cold start (a Reset that discards the flowsheet's saved converged state, forcing a full initial-guess search), the CoolProp/HEOS package's mixture

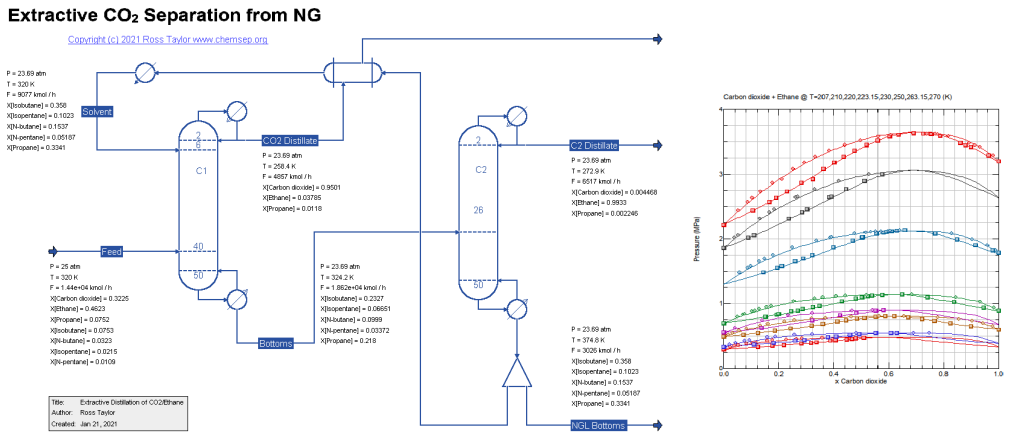


Figure 7.5: The CO2-from-NG extractive-distillation flowsheet (IECR 52, p.10780) in COFE.

density solver fails on an extreme trial composition that search probes. Solved from a warm state — either the flowsheet's saved state, or (confirmed, reproducible) a **reset -> solve with the same CoolProp package set to a cubic backend (PR/SRK) -> switch to HEOS -> solve** sequence — it converges cleanly with no errors or warnings. This is exactly the scenario documented in *Cold-start initialization on hard mixtures*, and this flowsheet is the worked example: both `.fsd` files here were solved warm (opened and solved from the saved state, as in normal COFE use — matching what a user actually does), not from a COFEStand batch reset. The native reference is ChemSep with the Predictive Peng-Robinson 78 (PPR78) EOS.

Table 7.3: CO2-from-NG: native ChemSep/PPR78 vs. CoolProp HEOS

Metric	Native	CoolProp	Difference
C1 reboiler / condenser duty [MW]	66.17 / 109.44	63.84 / 108.56	-3.5 / -0.8%
C2 reboiler / condenser duty [MW]	92.13 / 71.61	93.12 / 72.38	$+1.1$ / $+1.1\%$
Total reboiler duty [MW]	158.31	156.96	-0.8%
CO2 product purity [mol%]	95.00	93.36	-1.65 pt
Ethane in CO2 product [mol%]	3.50	5.93	$\times 1.7$
Ethane product purity [mol%]	99.54	97.73	-1.81 pt
CO2 in ethane product [mol%]	0.45	1.68	$\times 3.7$
CO2 recovery [%]	99.36	97.64	-1.72 pt
Ethane recovery [%]	97.45	95.68	-1.77 pt

The total reboiler duty agrees to 0.8%, but the columns do not agree individually: the C1 (extractive) reboiler falls by 3.5% while C2's duties rise by 1.1%. The **separation diverges much more**. Purities and recoveries move by 1.6–1.8 percentage points, and the cross-contamination that sets those purities moves by a factor of 1.7 (ethane in the CO_2 product) to 3.7 (CO_2 in the ethane product). This is the expected place for the two models to differ: the separation depends directly on the CO_2/ethane relative volatility near the azeotrope and on the solvent's effect on it, the most EOS-sensitive quantities in the flowsheet, and PPR78's group-contribution binary parameters and CoolProp's HEOS departure functions represent those binaries differently. The comparison shows the package delivering a different, internally consistent EOS into the column solve; it is not evidence that either model is the more accurate one for this system.

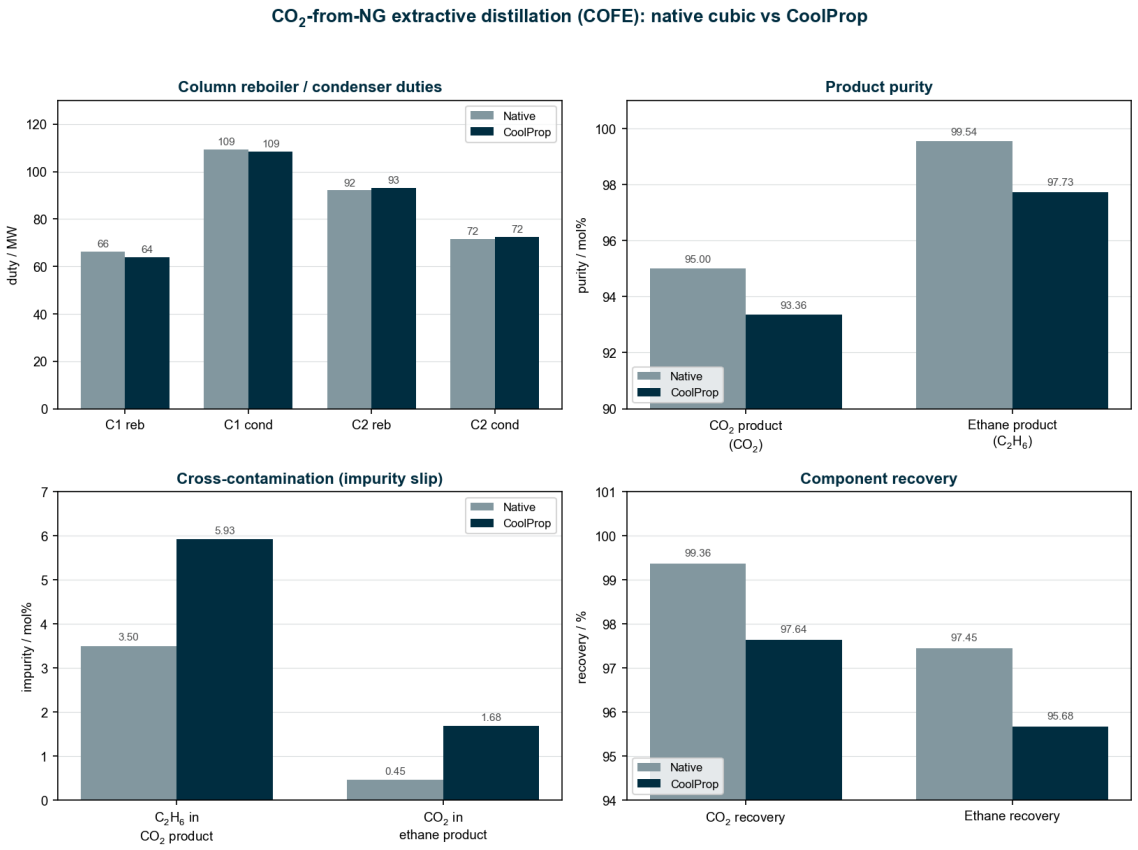


Figure 7.6: Column duties, product purity, cross-contamination, and recovery: native ChemSep/PPR78 vs CoolProp (HEOS).

7.4 Snøhvit: single mixed refrigerant, multi-stage compression

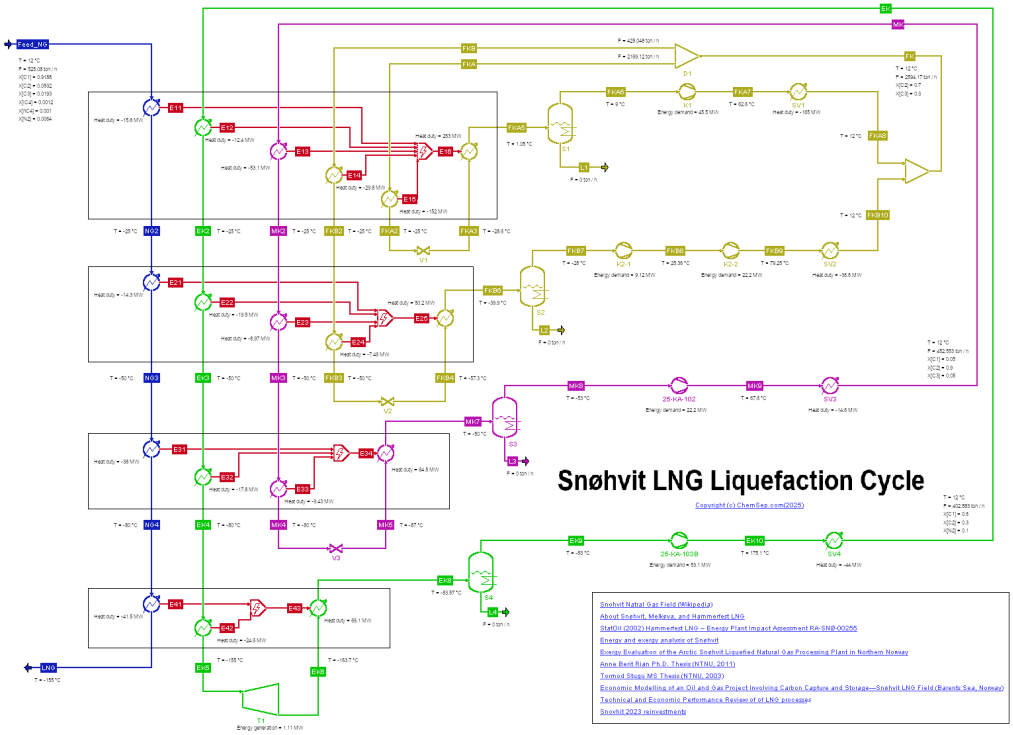


Figure 7.7: The Snøhvit single mixed-refrigerant LNG liquefaction flowsheet in COFE.

A natural gas feed (60 bar, 285.15 K, 8333.33 mol/s) is liquefied to LNG (118.15 K, 145.86 kg/s) against a single mixed-refrigerant cycle compressed across five stages (K1, K2-1, K2-2, 25-KA-102, 25-KA-103B) with a recompression turbine (T1) recovering work from a let-down. Feed and LNG product T/P/flow are confirmed **identical** specifications in both files — this is not assumed, it is read back from both solved flowsheets directly.

> **Note**
>
> **One compound-set difference, disclosed rather than hidden.** The native flowsheet carries a trace water compound (8 compounds: C1/C2/C3/nC4/iC4/N2/CO2/H2O); the CoolProp flowsheet's feed omits it (7 compounds). This predates the validation exercise and is a defensible modelling simplification in its own right — water is normally removed upstream of the liquefaction section by molecular-sieve dehydration in the real process, so a dehydrated feed is a reasonable compound set for the liquefaction-section property package to carry — but it does mean this comparison is not bit-for-bit identical the way TEALARC's is, and the differences below mix the compound-set difference with the EOS difference (ChemSep Peng-Robinson 76 vs HEOS). It is flagged here explicitly rather than presented as a clean match it is not.

Table 7.4: Snøhvit: native ChemSep/PR76 vs. CoolProp HEOS

Metric	Native	CoolProp	Difference
Total compression [MW]	142.28	143.25	+0.68%
Net train power [MW]	141.17	142.14	+0.68%
Specific power [kWh/t LNG]	268.9	270.7	+0.68%

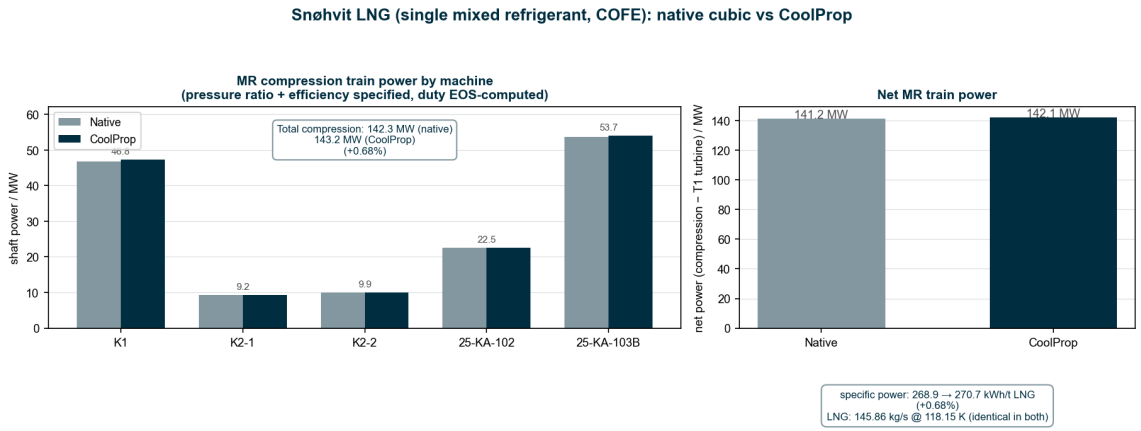


Figure 7.8: MR compression-train shaft power by machine and net train power: native ChemSep/PR76 vs CoolProp (HEOS).

Every machine in the five-stage train and the recompression turbine agrees to within about 1% (K1 +0.96%, K2-1 +0.23%, K2-2 +1.04%, 25-KA-102 +0.48%, 25-KA-103B +0.54%, T1 +0.67%), and the net-power and specific-power deltas match the total-compression delta. Both solves converge at the saved initial guess, but their warnings differ, and the differences are reported here:

- **Temperature-limited heat exchangers.** The native solve warns that HeaterCooler_19's duty is limited by its energy-stream temperature; the CoolProp solve warns on HeaterCooler_19 and HeaterCooler_29. These belong to the flowsheet's approximation of a multistream exchanger as separate energy-linked HeaterCoolers; the extra warning means the CoolProp case is one exchanger closer to its temperature limit.
- **Partition 1 (CoolProp only).** COFE reports a 100% mass-balance error on the units of partition 1, which includes the recompression turbine T1 and the compressor 25-KA-103B. The partition has no external feed (it is the closed refrigerant loop), and the absolute imbalance is 1.7×10^{-14} kg/s, so the percentage is a division by a zero feed rather than a real imbalance. COFE also notes that stream EK carries a mass flow that is high relative to the partition's (zero) feed and asks for the recycle to be checked; for a closed loop that is expected. The T1 and 25-KA-103B duties in the table agree with the native case to within 0.7%, which a non-converged loop would not give.

7.5 Air separation unit (ASU): cryogenic N2/O2/Ar distillation

A 100 kg/s air feed (N2/O2/Ar) is compressed in a three-stage inter-cooled main air-compression train (C1, C2, C3 — each specifying a pressure ratio and isentropic efficiency, so shaft power is EOS-computed from the enthalpy rise across each stage), cooled and liquefied with a turbo-expander providing the refrigeration, and separated in a cryogenic double column plus a crude-argon column into gaseous oxygen (G O2, 98.5 mol% O2), gaseous nitrogen (G N2), liquid argon (L Ar, 99.99 mol% Ar) and a nitrogen-rich waste stream. This is the low-temperature, tight-boiling-point case: N2, O2 and Ar boil within a few kelvin of one another (~77–90 K), so the column temperatures, relative volatilities and hence the product purities and compression power are a demanding test of the mixture EOS at cryogenic conditions.

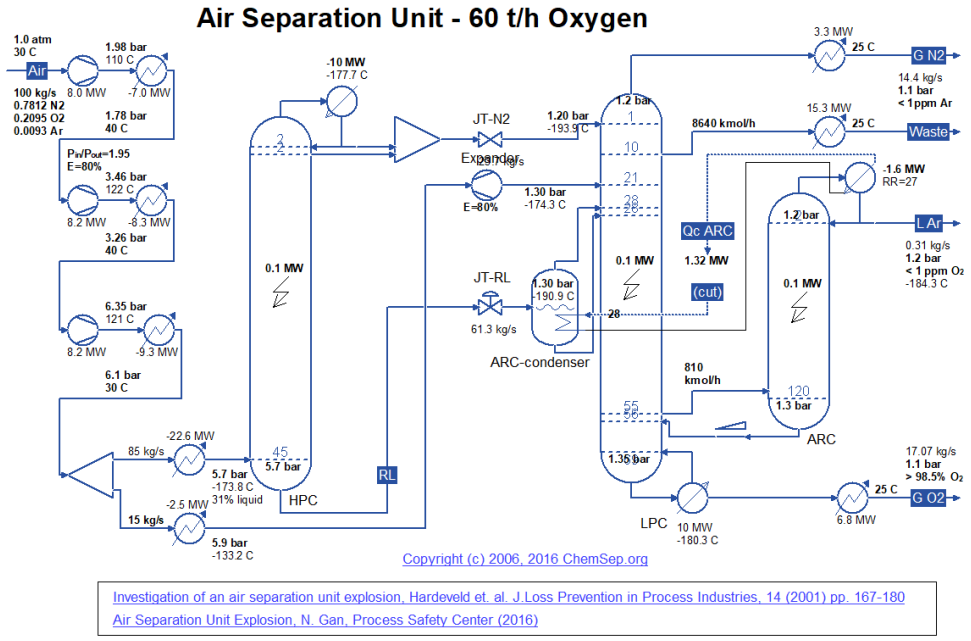


Figure 7.9: The CScasebook cryogenic air-separation flowsheet in COFE (ChemSep case study): three-stage main air compression, turbo-expander, high- and low-pressure columns (HPC/LPC) and the crude-argon column (ARC), producing gaseous oxygen, gaseous nitrogen and liquid argon.

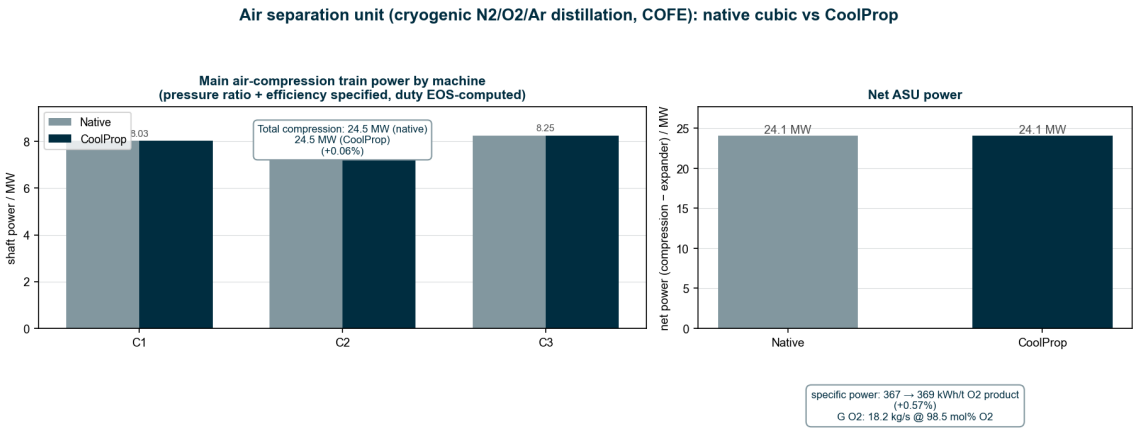


Figure 7.10: Main air-compression train shaft power by machine, net ASU power and specific power per tonne of oxygen product: native ChemSep/PR76 vs CoolProp (HEOS).

Table 7.5: ASU: native ChemSep/PR76 vs. CoolProp HEOS

Metric	Native	CoolProp	Difference
C1 / C2 / C3 compressor power [MW]	8.030 / 8.221 / 8.248	8.033 / 8.226 / 8.256	+0.04 / +0.06 / +0.09%
Total compression [MW]	24.50	24.52	+0.06%
Expander recovered [MW]	0.420	0.421	+0.15%
Net power (compression - expander) [MW]	24.08	24.09	+0.06%
G O2 flow [kg/s]	18.24	18.15	−0.50%
G N2 flow [kg/s]	13.72	13.69	−0.23%
L Ar flow [kg/s]	0.328	0.331	+1.0%
Specific power [kWh/t O2 product]	366.8	368.9	+0.57%

The compression train, the expander and the net power agree to better than 0.2%. The product **flows** differ more (-0.5% gaseous oxygen, +1.0% liquid argon), and because the specific power is divided by the oxygen flow it moves by +0.57%. The product **purities** are not compared: the gaseous-oxygen (98.5 mol%) and liquid-argon (99.99 mol%) purities are column specifications, so both packages meet them by construction, and the EOS difference appears in the flows needed to meet them instead. The largest relative delta falls on the smallest stream (liquid argon, 0.33 kg/s), where a small difference in the $N_2/O_2/Ar$ relative volatilities is proportionally largest.

7.6 CO_2 compression, dehydration and liquefaction (CCS)

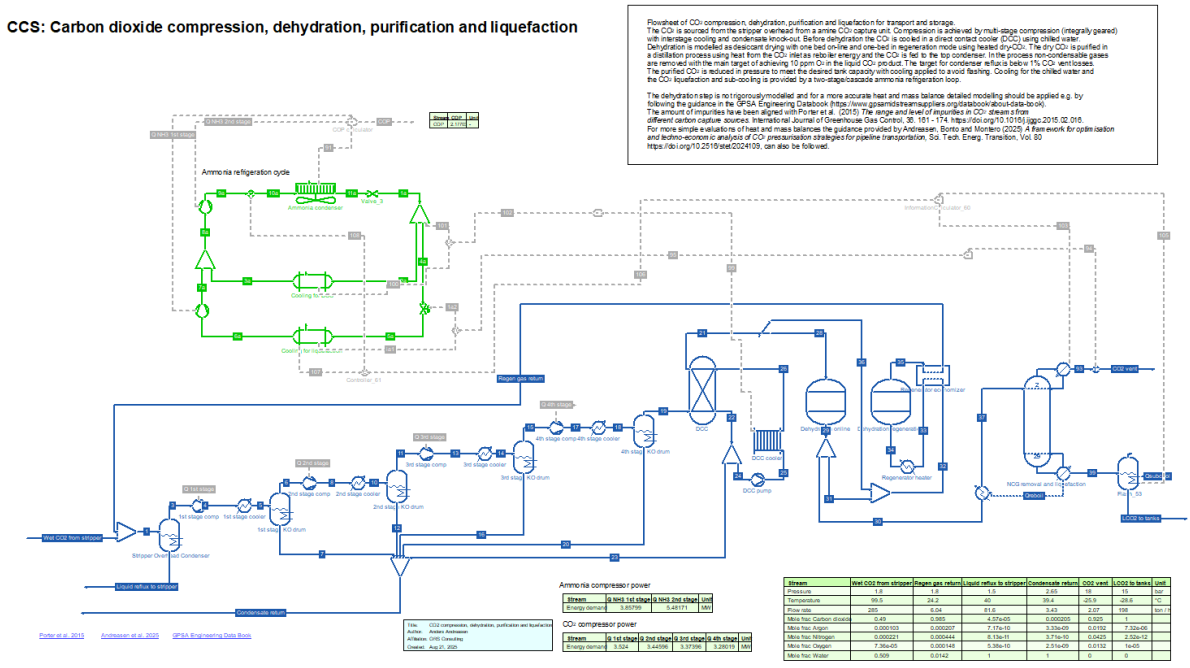


Figure 7.11: COFE flowsheet of the CO_2 compression, dehydration, purification and liquefaction (CCS) process: multi-stage integrally-geared CO_2 compression with interstage cooling and knock-out, a direct-contact cooler, desiccant dehydration, distillation-based non-condensable-gas removal and liquefaction, with a two-stage cascade ammonia refrigeration loop (green). Diagram: `cocosimulator.org/downloads/CO2_compression_liquefaction_Andreasen_2025.png`.

The author's CO_2 compression, dehydration, purification and liquefaction design: multi-stage integrally-geared CO_2 compression with interstage cooling and knock-out, a direct-contact cooler (DCC), desiccant dehydration, distillation-based non-condensable-gas removal and liquefaction, served by a two-stage cascade ammonia refrigeration loop. The feed is a wet, impure CO_2 stream (CO_2/Ar/N_2/O_2/H_2O) and the product is liquid CO_2 at 15 bar delivered to tanks. This is the CCS case of the set, and the only one in which the switched package must span the full range from bone-dry dense-phase CO_2 to near-pure liquid water inside a single compound set.

Unlike the other five flowsheets, this is **not** a whole-flowsheet package swap. COFE binds a property package to a *material type*, and here only the `Carbon dioxide` type — 39 of the 60 material streams — switches from the native cubic to CoolProp. The DCC circulating water loop (7 streams, its own `DCC` type) and the ammonia refrigeration loop (14 streams) stay on the native cubic in **both** cases, the ammonia loop thereby acting as a near-control. The CoolProp case also carries three `StreamConverter` units at the material-type boundary, so the two files are not structurally identical (60 streams / 50 units vs 57 / 47); quantities that straddle that boundary — notably the water condensate — inherit a converter re-flash and are the least clean rows in the table below.

Both files are solved from their saved state (no reset), as in normal COFE use; the CoolProp case does not solve from a reset because the reset discards the initial guess of the controller (`Controller_17`) on the stream feeding it. Unlike the other five CoolProp files, this one has the CoolProp package set to the **Native** PH/PS strategy (*PH/PS flash strategy*), as saved by the author. The native reference is COCO TEA with Soave-Redlich-Kwong.

Both solves log warnings that the table must be read with. In both files COFE warns that the DCC feed pressure is below the column pressure, the DCC bottom-product PVF flash falls back to a TP re-flash, and the DCC pump reports vapour at its inlet. Separator flashes also fall back to a TP flash, more often on the native side: there, all four knock-out drums and the stripper overhead condenser fail their temperature post-check, and TEA reports trivial-solution failures of its inside-out TP flash. On the CoolProp side only the 4th-stage knock-out drum falls back (a failed dew-point flash). The water-condensate rows below come from these drums and the DCC.

Table 7.6: CO_2 compression/liquefaction: native cubic vs. CoolProp

Metric	Native	CoolProp	Difference
CO_2 compressor train, 4 stages [MW]	13.624	13.611	-0.10%
NH_3 refrigeration (cubic in both) [MW]	9.340	9.503	$+1.75\%$
Total shaft power [MW]	22.964	23.114	$+0.66\%$
Dehydration reboiler duty [MW]	1.417	0.956	-32.5%
LCO_2 product: CO_2 [mol%]	99.9983	99.9977	-0.0006 pt
LCO_2 product: Ar [ppm]	7.32	13.44	$+84\%$ (small abs.)
LCO_2 product: N_2 [ppm]	~ 0	0.016	trace either way
LCO_2 product flow [kg/s]	54.977	54.951	-0.05%
CO_2 vent: CO_2 [mol%]	92.51	92.36	-0.15 pt
CO_2 vent: molar flow [mol/s]	13.36	13.00	-2.7%
Condensate: dissolved CO_2 [ppm]	205	2453	$\times 12$
Condensate return: molar flow [mol/s]	52.80	67.12	$+27\%$

The EOS choice separates the results where the underlying phase equilibria differ:

- **The CO_2 compressor train is essentially EOS-independent.** It agrees to 0.10 % and the product flow to 0.05 %. The total shaft power moves more (+0.66 %), but that comes from the ammonia

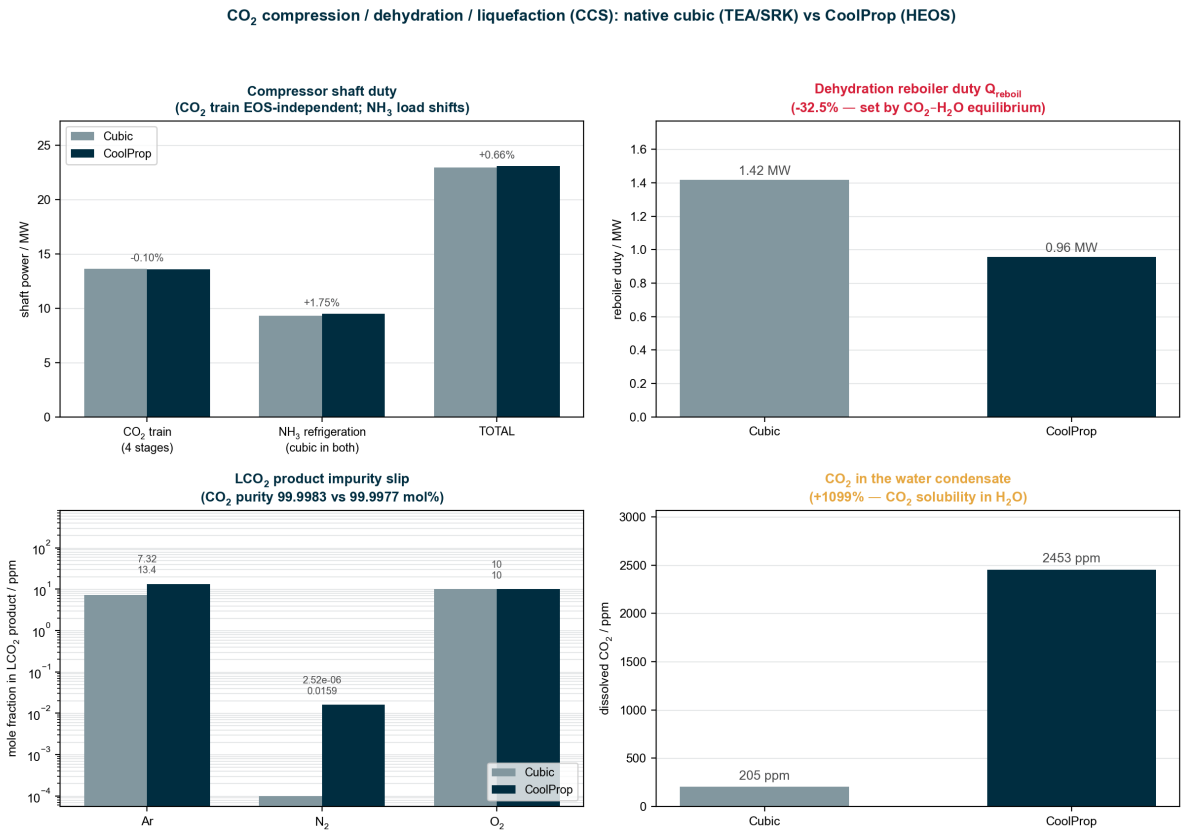


Figure 7.12: CO_2 compression / dehydration / liquefaction solved in COFE with the native cubic (TEA/SRK) and with CoolProp (HEOS): compressor shaft duty (top left), dehydration reboiler duty (top right), LCO_2 product impurity slip (bottom left; the native N_2 bar is drawn at a floor of 10^{-4} ppm, its labelled value being 2.5×10^{-6} ppm) and CO_2 dissolved in the water condensate (bottom right).

refrigeration loop (+1.75 %), which is on the same TEA/SRK package in both files. Its duty changes because the process side it cools (the CO_2 streams) is computed by a different package, so the refrigeration load it is asked to remove changes; the loop's own thermodynamics do not. The total is therefore not a measure of the CoolProp package's agreement, and the CO_2 train is.

- **The dehydration reboiler duty is highly EOS-sensitive**, falling by a third (1.42 → 0.96 MW). The stripper duty is set by the CO_2–H_2O equilibrium, so it is the single most model-dependent number in the flowsheet.
- **The water condensate carries ~12× more dissolved CO:sub:`2`** (205 → 2453 ppm) with a 27 % larger return flow — the direct signature of a different CO_2–H_2O mutual-solubility model. This row straddles the package boundary and should be read as indicative rather than a like-for-like comparison. It is also the row most sensitive to the CoolProp flash revision: solved on the previous bundled build (`c10fc2ac`) during the 2026-10-26 update, it gave 1148 ppm and 77.7 mol/s and needed TP-flash fallbacks in all four KO drums; the current build's liquid-liquid stability probe (CoolProp PR #3473) leaves one (the 4th stage). Every other row moved by less than 0.4 %. The `c10fc2ac` solve is not committed; only the current results are.
- **Product purity is preserved.** CoolProp slips marginally more Ar (13.4 vs 7.3 ppm) and a trace of N_2, but the LCO_2 product remains at 99.998 mol% CO_2 — comfortably inside any CCS specification.

The author's companion GERG-2008 property-package study of this same flowsheet (a separate technical reference, not reproduced here) gives a further point of comparison. There, stock GERG-2008 and an EOS-CG-style "mutant" variant give reboiler duties of 0.72 and 0.96 MW, Ar slips of 117 and 13 ppm, and N_2 slips of 94 and 0.02 ppm. CoolProp (0.96 MW, 13.4 ppm, 0.016 ppm) lands close to the mutant variant. For a CCS purity specification that difference is decisive, and it points to the CO_2–Ar and CO_2–N_2 binary models rather than the pure-fluid EOS as the driver. These GERG-2008 figures come from that study and are not re-derived from the files committed here.

7.7 Summary

Table 7.7: Headline agreement across the six validation flowsheets

Flowsheet	Reference / compounds	Headline metric	Native -> CoolProp	Solve (native; CoolProp)
CHP	IAPWS-IF97; pure water	net shaft power	$< 0.01\%$	warm, at guess; `/r`, 6 it.
TEALARC	TEA/SRK; 5 compounds	total compressor power	-2.0% (machines -0.2 to -3.7%)	`/r`, 1 it.; `/r`, 1 it.
CO2-from-NG	ChemSep/PPR78; 7 compounds, near-azeotrope	product purity / recovery	-1.6 to -1.8 pt	warm, at guess; warm, at guess
Snøhvit	ChemSep/PR76; 8 vs 7 compounds	net MR train power	$+0.7\%$	`/r`, at guess; `/r`, at guess
ASU	ChemSep/PR76; N2/O2/Ar	net power	$+0.06\%$	`/r`, 8 it.; `/r`, 5 it.
CO2 compression (CCS)	TEA/SRK; partial swap, 5 compounds	CO2 train shaft power / LCO2 purity	-0.10% / -0.0006 pt	warm, at guess; warm, at guess

All twelve files solve with `SOLVE: OK` on the documented build, with the compound-order-aligned package (*Compound-order alignment to the Material Object*). The Snøhvit CoolProp solve carries a closed-loop mass-balance warning and one more temperature-limited exchanger than the native one (*Snøhvit: single mixed refrigerant, multi-stage compression*), and the CO_2 compression solves carry flash-fallback warnings on both sides (*CO2 compression, dehydration and liquefaction (CCS)*). Every comparison metric differs from the native reference by an amount attributable to the different equations of state (and, for Snøhvit, the different compound set) — never by the order-of-magnitude, wrong-phase discrepancies that the pre-fix compound-order bug produced. That distinction is the actual claim of this chapter: not that CoolProp and a native cubic package agree exactly (they should not — they are different equations of state), but that the property-package interface delivers whichever EOS is selected *correctly* into a converged, real flowsheet solve. One flowsheet (CO2-from-NG) additionally surfaced a genuine, documented cold-start limitation of the HEOS mixture solver with a confirmed, practical workaround (*Cold-start initialization on hard mixtures*) — included here deliberately rather than quietly worked around, since a

limitation with a known fix is exactly the kind of finding this validation exercise exists to surface.

Part IV

Appendices

Chapter 8

Appendix: Options and Property Identifiers

8.1 Configuration options

Set via the `Edit()` GUI (*Architecture*) and persisted with the flowsheet (*Design Decisions*).

Option	Values	Notes
Equation of state	PR / SRK / HEOS	*Equation-of-state / backend selection*. One EOS per package instance.
PH/PS flash strategy	C# (default) / Native	*PH/PS flash strategy*. Override for diagnostics: `COOLPROPCO_PHPS=native` / `=csharp` env var.
Compound selection	any subset of the CoolProp fluid catalogue	Realigned to the Material Object's own order by CAS before every `CalcEquilibrium` and `CalcSinglePhaseProp` call — *Compound-order alignment to the Material Object*. (`CalcTwoPhaseProp` and `CalcAndGetLnPhi` use the order already in force.)
Binary interaction parameters (kij)	per-pair override	Keyed `"CAS1|CAS2"` in the persisted stream; applied via `AbstractState_set_binary_interaction_double(..., "kij", value)`. Effective on **PR and SRK only**. HEOS has no `kij` parameter (its binary parameters are $\beta_T, \gamma_T, \beta_V, \gamma_V$ and F_{ij}), so CoolProp rejects the call and the package ignores it silently: kij overrides have **no effect** on the default HEOS backend.

8.2 Diagnostic environment variables

Not part of the persisted flowsheet state — process-level, for troubleshooting and CI.

Variable	Values	Effect
COOLPROPCO_LOG	info / debug / all, off / none / silent, unset	Verbosity of coolpropco.log (next to the assembly). Unset = errors only (the hot-path default — verbose logging touches disk on every flash).
COOLPROPCO_PHPS	native / csharp	Overrides the configured/persisted PH/PS strategy (*PH/PS flash strategy*) without touching flowsheet state.

8.3 Supported CAPE-OPEN property identifiers

Only identifiers in the CAPE-OPEN 1.1 standard registry (§7.5.5/§7.5.6) may be advertised — a PME (COFE in particular) validates every entry in `GetSinglePhasePropList`/`GetTwoPhasePropList` against that registry and raises "no such property" on anything else. CoolProp-only quantities without a standard CAPE-OPEN identifier (ideal-gas C_v, bulk modulus, isobaric expansion coefficient, isentropic exponent) are therefore **not** advertised, even though the engine can compute them internally.

Table 8.1: Single-phase properties (`CalcSinglePhaseProp`)

Identifier	Source
enthalpy, entropy	CoolProp Hmolar/Smolar (always molar basis)
internalEnergy, gibbsEnergy, helmholtzEnergy	CoolProp Umolar/Gmolar/Helmholtzmolar
heatCapacityCp, heatCapacityCv	CoolProp Cpmolar/Cvmolar
volume, density	CoolProp Dmolar (reciprocal for volume)
compressibilityFactor	CoolProp Z
fugacityCoefficient, logFugacityCoefficient	CoolProp fugacity_coefficient per compound. For an exactly pure composition the pure compound's value comes from its pure-fluid handle, and the absent compounds' infinite-dilution values from the phase's mixture handle (1.0 only if that state cannot be computed)
viscosity, thermalConductivity	CoolProp native, else *Transport-Property Fallbacks* fallback correlations
speedOfSound	CoolProp speed_of_sound
molecularWeight	CoolProp M of the evaluated state (mole-fraction-weighted molar mass), converted to kg/kmol
idealGasHeatCapacity, idealGasEnthalpy, idealGasEntropy	CoolProp Cp0molar/ideal-gas reference state (needed by COFE for reference-state harmonisation)
jouleThomsonCoefficient	$(\partial T/\partial P)_H$ from CoolProp's analytic first_partial_deriv
compressibility	**Isothermal compressibility** [1/Pa] — note the CAPE-OPEN standard identifier is compressibility, not "isothermalCompressibility"

Table 8.2: Two-phase properties (`CalcTwoPhaseProp`)

Identifier	Source
`kvalue`, `logKvalue`	$K_i = z_i^{(1)}/z_i^{(2)}$: the ratio of the mole fractions the Material Object holds for the first and second phase labels the PME passes (normally vapour then liquid, giving y_i/x_i)
`surfaceTension`	CoolProp's native two-phase surface tension (bubble-point state), else the Brock-Bird fallback (*Surface Tension*)

Table 8.3: Compound constants (`GetCompoundConstant`)

Identifier	Notes
`molecularWeight`	kg/kmol (converted from CoolProp's kg/mol)
`criticalTemperature`, `criticalPressure`, `criticalVolume`	
`criticalCompressibility`	Computed from $P_c V_c/(RT_c)$ when V_c is known
`acentricFactor`, `normalBoilingPoint`	
`casRegistryNumber`, `chemicalFormula`	The CAS number is also the key used for compound-order alignment (*Compound-order alignment to the Material Object*) and kij lookup

Only `idealGasHeatCapacity` is offered as a T-dependent compound property; no P-dependent compound properties are provided (`GetPDependentPropList` returns empty/`VT_EMPTY`).

Chapter 9

Bibliography

Bibliography

[1] CO-LaN. CAPE-OPEN thermodynamic and physical properties, version 1.1. https://www.colan.org, 2011. CO Thermo 1.1 specification (interfaces implemented by the property package).

[2] Ian H. Bell, Jorrit Wronski, Sylvain Quoilin, and Vincent Lemort. Pure and pseudo-pure fluid thermophysical property evaluation and the open-source thermophysical property library CoolProp. *Industrial & Engineering Chemistry Research*, 53(6):2498–2508, 2014. HEOS backend; speed baseline and VLE cross-check oracle. doi:10.1021/ie4033999.

[3] Eric W. Lemmon and Reiner Tillner-Roth. A Helmholtz energy equation of state for calculating the thermodynamic properties of fluid mixtures. *Fluid Phase Equilibria*, 165(1):1–21, 1999. doi:10.1016/S0378-3812(99)00262-9.

[4] Eric W. Lemmon and Richard T. Jacobsen. A generalized model for the thermodynamic properties of mixtures. *International Journal of Thermophysics*, 20(3):825–835, 1999. doi:10.1023/A:1022627001338.

[5] Oliver Kunz and Wolfgang Wagner. The GERG-2008 wide-range equation of state for natural gases and other mixtures: an expansion of GERG-2004. *Journal of Chemical & Engineering Data*, 57(11):3032–3091, 2012. 21-component multi-fluid Helmholtz model; tabulates all required Helmholtz-derivative relations. doi:10.1021/je300655b.

[6] A.L. Lee, M.H. Gonzalez, and B.E. Eakin. The viscosity of natural gases. *Journal of Petroleum Technology*, 18(8):997–1000, 1966. doi:10.2118/1340-PA.

[7] John Lohrenz, B.G. Bray, and C.R. Clark. Calculating viscosities of reservoir fluids from their compositions. *Journal of Petroleum Technology*, 16(10):1171–1176, 1964. doi:10.2118/915-PA.

[8] Karen S. Pedersen and Peter L. Christensen. *Phase Behavior of Petroleum Reservoir Fluids*. CRC Press, 2007.

[9] F. Herning and L. Zipperer. Calculation of the Viscosity of Technical Gas Mixtures from the Viscosity of Individual Gases. *Gas- und Wasserfach*, 79:69–73, 1936.

[10] Farhad Gharagheizi, Poorandokht Ilani-Kashkouli, Mehdi Sattari, Amir H. Mohammadi, Deresh Ramjugernath, and Dominique Richon. Development of a QSPR model for determination of thermal conductivity of non-electrolyte organic compounds. *AIChE Journal*, 59(5):1702–1708, 2013. doi:10.1002/aic.13938.

[11] James R. Brock and R. Byron Bird. Surface tension and the principle of corresponding states. *AIChE Journal*, 1(2):174–177, 1955. doi:10.1002/aic.690010208.

[12] Wolfgang Wagner and Andreas Pruß. The IAPWS formulation 1995 for the thermodynamic properties of ordinary water substance for general and scientific use. *Journal of Physical and Chemical Reference Data*, 31(2):387–535, 2002. doi:10.1063/1.1461829.